

Degree Programme

Systems Engineering

Major Infotronics

Bachelor's Thesis

Diploma 2026

Paulo Junior Gomes Costa

Probes for DOTS (Distributed OT Test Suite)

Development of a test probe for performing
distributed functional and security-related tests
in OT environments.

Professor

HEI-Vs, Prof. Rico Steiner, rico.steiner@hevs.ch

Expert

HOOC AG, Gil Beauge, gil.beauge@hooc.ch

Submission date

Aug 14, 2026



Degree program SYND	Academic year 2025-26	Nbr Thesis IT/2026/10
Mandant ☒ HES—SO Valais-Wallis ☐ Industry ☐ Partner institution	Student Gomes Costa Paulo Junior Professor Steiner Rico	Place of execution ☒ HES—SO Valais-Wallis ☐ Industry ☐ Partner institution
Confidential work ☐ Yes ☒ No	Expert (full name, E-Mail, Address) Beauge Gil	

Title

Development of a test probe as part of an generic test suite for performing distributed functional and security-related tests in OT environments.

Description

Current standards for OT environments require numerous functional and security-related tests, which must be defined as well as repeated and logged each time the system is modified. Preparing the test environment and conducting these tests is a complex procedure, as manipulations must be performed at various points of the system and the effects need to be observed.

To solve this problem, a system consisting of two parts comes into play:

- Probes in the form of an embedded system are to be developed. Placing these probes in the test environment allows to perform test operations such as but not limited to ping, HTTP requests, capturing Ethernet traces, reading and writing I/Os, etc.
- A central test management and coordination system (coordinator), which is NOT part of this project, performs predefined tests by interacting with the probes.

Having such a system allows automation of complex test procedures, with reproducible and comparable results.

The probe shall provide the following functions:

- Getting registered to a coordinator
- Sending the list of capabilities to the coordinator
- Receiving commands from the coordinator, executing them and sending the results back to the coordinator
- Local logging of the executed commands

Individual project (Pr4) objectives

- Study and comparison of existing, similar projects or products (commercial and/or open source).
- Evaluation of possible system architectures
- Defining the technologies, tools, libraries, and hardware platform for realizing the project
- Purchasing of the hardware as well as conducting a fundamental proof of concept
- Defining the interface (capabilities, technologies & protocol) between the probe to be developed and the management system
- Defining the scope of the bachelor thesis and elaboration of the system specification
- Establish a detailed Gantt diagram and work plan for the Bachelor thesis, including milestones and deliverables.

Bachelor's thesis (TB) objectives

- Documentation: produce the complete bachelor thesis report, including system architecture, experiments, evaluation results, and user/technical documentation.
- Design and implementation of the probe according to the specifications and results derived from PR4.
- Performing functional, performance and stability tests
- Building of a sample test environment for demonstrating the system's capabilities
- Optional: Integrate the work with a coordinator system and demonstrate interoperability of probes and coordinator.

Deadlines

Start of individual project (Pr4)

16.02.2026

Submission of the Pr4 report

01.05.2026, 12:00

Presentation of the individual project (Pr4)

Week 19 (04-08.05.2026)

Start of bachelor's thesis

18.05.2026

Submission of the final report

14.08.2026, 12:00

Exhibitions and pitch of bachelor's theses

28.08.2026 – HEI

31.08.2026 – Monthey

03.09.2026 – Visp

Oral defense

Week 35-37 (24.08-11.09.2026)

Signature or visa

Head of the major



¹ Student



¹ By signing, the student agrees to comply with guideline DI.1.2.02.07 "Bachelor's Thesis".



Bachelor's Thesis

| 2026 |

Degree Programme
Systems Engineering

Major
Infotronics

Professor
Prof. Rico Steiner
rico.steiner@hevs.ch

Probes for DOTS (Distributed OT Test Suite)

Graduate

Paulo Junior Gomes Costa

Objective

The objective of this work is to develop and implement an embedded probe capable of performing various test operations within OT environments, collecting their results, and communicating them to a central coordinator.

Methods | Experiences | Results

Operational Technology (OT) environments require numerous functional and security-related tests to be defined, repeated, and logged whenever a system is modified. Preparing and conducting these tests can be complex, as operations must be performed at different points of the system and their effects observed. This work focuses on the development of configurable probes in the form of embedded systems for distributed OT testing.

The probe is a configurable and remotely controlled device capable of performing various commands when deployed in an OT environment. It connects to a central coordinator and advertises its capabilities and configurable parameters, allowing it to be adapted to different test environments. The coordinator can then send commands or configuration updates, while the probe executes the requested commands and returns their results. Communication relies on MQTT, while local logging provides traceability of executed operations.

The solution was validated on a physical test bench composed of three probes and one Device Under Test. A test suite was conducted to evaluate the probe's operation and interoperability with the DOTS coordinator.

Information about this report

Contact Information

Author: Paulo Junior Gomes Costa

Bachelor Student

HEI-Vs

E-Mail: paulojun.gomescos@hes-so.ch

paulojuniorgomescosta@gmail.com

Declaration of honor

I, undersigned, hereby declare that the work submitted is the result of a personal work. I certify that I have not resorted to plagiarism or other forms of fraud. All sources of information used and the author quotes were clearly mentioned

Use of generative AI

AI tools were used as support throughout the development of this project. All technical content, project decisions, and presented results originate from the author. All generated information was reviewed and verified before being used.

Place, date Sion, 14.08.2026

Signature



Contents

Acknowledgements	1
Abstract	2
1 Introduction	3
1.1 Context Problem	3
1.2 Objectives	3
1.3 Structure of this report	3
2 Sustainability integration	5
3 Analysis	6
3.1 Hardware requirements	7
3.2 NanoPi R5S	7
3.3 Odroid H4 + Netcard 3	8
3.4 Raspberry Pi 4	9
3.5 Hardware selection	9
4 Design	10
4.1 Overview of the dual-environement system	11
4.2 Requirements Specification	12
4.3 Architecture of the probe	14
4.4 Librairies used	16
4.5 Interface with the coordinator	17
4.5.1 probes/announce	17
4.5.2 probes/discover	18
4.5.3 probes/{probe_id}/control	19
4.5.4 probes/{probe_id}/command	20
4.5.5 probes/{probe_id}/response	20
4.6 Deployment	21
5 Implementation	22
5.1 Implementation of the probe workflow	23
5.2 Broker detection and connection	24
5.3 Implementation of the communication with the coordinator	24
5.3.1 Discovery request and status	25
5.3.2 Configuration request	26
5.3.3 Command request	27
5.4 Implementation of the configuration paramaters list	28
5.4.1 Format of the configuration parameters groups	28
5.4.2 Format of the configuration parameters list	29
5.5 Implementation of the capabilities list	30
5.6 Configuration of the interfaces	32
5.6.1 Configuration of IP address	33
5.6.2 Creation of a bridge	34
5.6.3 Configuration of a DHCP server	35
5.6.4 Issue encountered	35
5.7 Implementation of the capabilities	36

5.7.1	Ping	36
5.7.2	Capture	37
5.7.3	Port scan	38
5.7.4	HTTP request	38
5.7.5	TCP request	39
5.7.6	TCP echo server	39
5.7.7	UDP request	40
5.7.8	UDP echo server	41
5.7.9	Send pcap	41
6	Proof of concept of net.resolver	43
6.1	Introduction	44
6.2	Explanation of the method	44
6.3	Results	45
6.4	Futur use	45
7	Validation	46
7.1	Testbench	47
7.2	Functionnal tests	48
7.3	Limit tests	50
7.3.1	Capture size limit	50
7.3.2	Parallel function limit	51
7.4	Conclusion	53
8	Conclusion	54
8.1	Project summary	54
8.2	Comparison with the initial objectives	54
8.3	Encountered difficulties	54
8.4	Future perspectives	55
	Glossary	56

Figures

Figure 1	Nano Pi R5S ¹	7
Figure 2	Odroid H4 ²	8
Figure 3	Netcard 3 ³	8
Figure 4	The CNet box (Raspberry Pi 4) ⁴	9
Figure 5	Diagram of the dual-environement system	11
Figure 6	Architecture of the probe	14
Figure 7	State machine that implement the probe workflow	23
Figure 8	Sequence diagram - Discovery request and status heartbeat	25
Figure 9	Sequence diagram - Control request	26

¹NanoPi R5S image[1]

²Odroid H4 image[2]

³Odroid H4 image[3]

⁴The image of the CNet box comes from a CNet laboratory

Figure 10	Sequence diagram - Command request	27
Figure 11	Test bench	47
Figure 12	Evolution of the execution time	52

Tables

Table 1	Responsability of the main packages of the probe	15
Table 2	Responsability of the internal packages of the probe package	15
Table 3	Libraries used in the probe	16
Table 4	Fields description of a configuration group format	29
Table 5	Fields description of a configuration parameter format	30
Table 6	Fields description of the Capability format	31
Table 7	Fields description of the CapabilityParams format	32
Table 8	Configuration parameters for a test interface	33
Table 9	Configuration parameters for the coordination interface	34
Table 10	Configuration parameters for the DHCP server configuration	35
Table 11	Ping execution parameters	36
Table 12	Ping results	37
Table 13	Capture execution parameters	37
Table 14	Capture results	37
Table 15	Portscan execution parameters	38
Table 16	Portscan results	38
Table 17	HTPP execution parameters	38
Table 18	HTPP results	39
Table 19	TCP execution parameters	39
Table 20	TCP results	39
Table 21	TCP echo server execution parameters	40
Table 22	TCP echo server results	40
Table 23	UDP execution parameters	40
Table 24	UDP results	41
Table 25	UDP echo server execution parameters	41
Table 26	UDP echo server results	41
Table 27	PCAP execution parameters	42
Table 28	Summary of functional test validations	50

Acknowledgements

I would like to thank my professor, Prof. Rico Steiner, for his continuous availability, guidance, and support throughout this Bachelor's thesis. His advice, feedback, and willingness to discuss the challenges encountered during the project were particularly valuable.

I would also like to thank HOOC AG for welcoming us during this project and for taking the time to discuss potential features and improvements that could be integrated into the system. I would also like to thank them for providing the hardware used in the test bench, which made it possible to validate the solution in a realistic environment.

Finally, I would like to thank everyone who helped or advised me throughout this Bachelor's thesis.

Abstract

Operational Technology (OT) environments require numerous functional and security tests to ensure their proper operation. These tests must be defined, repeated, and recorded whenever the system is modified to ensure traceability. However, preparing the test environment and conducting these tests is a complex procedure, as different operations must be performed at multiple points of the infrastructure and their effects must be observed. This work therefore focuses on the development of distributed probes that enable these test operations to be performed.

This Bachelor's thesis presents the development of a distributed probe designed to test OT environments. The probe is a configurable and remotely controlled device capable of performing various network tests, including connectivity tests, traffic capture and transfer, port scanning, HTTP requests, and TCP and UDP communications.

The probe was designed to operate in two distinct environments: a known configuration environment and a potentially changing test environment. This separation provides the probe with a known reference environment from which it can be configured to adapt to different test networks.

It supports different network configurations, including static and dynamic IP addressing of its interfaces as well as transparent bridge mode. Using the MQTT protocol, it communicates with a coordinator to receive commands and configuration parameters, execute the requested tests, and return their results.

Finally, the probe provides the coordinator with the definitions of its commands and configuration parameters, together with all the metadata required to dynamically generate the coordinator's interface. This approach allows new functionalities to be added to the probe without requiring modifications to the coordinator, making the architecture more flexible and extensible.

The probe was tested on a test bench containing three probes and one DUT (Device Under Test) to evaluate its various functionalities, performance and interoperability with the DOTS coordinator.

Keywords:

OT, MQTT, Distributed probe, HEI-Vs, Systems Engineering, Infotronics

1 | Introduction

1.1 Context Problem

OT environments play a critical role in the industry, as they are responsible for controlling and monitoring critical processes. OT systems interact directly with the physical world and are therefore subject to strict security and reliability requirements.

To ensure the proper operation of these systems, numerous functional and security tests must be carried out. These tests need to be defined, executed in a repeatable manner, and documented whenever the system is modified. However, implementing such tests remains a complex task. It often requires manual intervention on different network devices, as well as the observation of the effects produced at multiple points within the installation. This approach is time-consuming, difficult to reproduce, and increases the risk of human error.

To resolve this issue, the Distributed OT Test Suite project proposes an architecture designed to automate test campaigns. This architecture relies on a central coordination system responsible for planning and orchestrating test scenarios, as well as probes deployed directly within the OT environment. These probes execute the test commands asked by the coordinator, perform various test operations, and return the results obtained.

1.2 Objectives

The objective of this Bachelor's thesis is to develop a probe that can be deployed within an OT environment to perform distributed test operations. The probe must be capable of:

- Registering itself with a coordinator, allowing the system to identify and manage the different probes deployed within the OT environment.
- Announcing the list of capabilities it implements, enabling the coordinator to determine which test operations can be performed by each probe.
- Providing the coordinator with a list of its configurable parameters, allowing its network and operational configuration to be modified remotely according to the requirements of the test environment. These configuration changes must be stored locally so that they are preserved across restarts.
- Receiving and executing commands sent by the coordinator, then transmitting the results of the executed tests back to the coordinator.
- Maintaining a local log of the commands executed by the probe.

1.3 Structure of this report

This report is structured into several sections covering the different stages of the thesis. First, the Analysis presents the evaluation of the different hardware options considered for the probe. The Design section then describes the dual environments in which the probe is deployed, its overall architecture, and its communication with the coordinator. The Implementation section details the development of the different components required to meet the defined requirements. A Proof of Concept section presents an additional feature that could potentially be integrated into the system in the future. The Validation section

evaluates the developed solution through tests performed on a physical test bench. Finally, the Conclusion summarizes the work and discusses the main results and possible future improvements.

2 | Sustainability integration

Operational Technology (OT) environments are used to control and monitor physical processes in many sectors, including industry, energy, and transportation. A failure in these systems can have significant consequences for service availability, production, as well as the safety of facilities and people. It is therefore essential to be able to test their behaviour under controlled conditions in order to identify potential issues before they occur in a real environment.

This thesis focuses on the development of probes for testing OT environments. The objective is to provide a flexible tool capable of verifying the behaviour and robustness of these infrastructures under different conditions. The project therefore aims to facilitate their validation and contribute to improving their reliability and resilience.

In this context, the project can be related to several Sustainable Development Goals (SDGs) defined by the United Nations. The most direct connection is with **SDG 9 – Industry, Innovation and Infrastructure**, which promotes reliable, sustainable, and resilient infrastructure as well as innovation. Developing a solution that enables OT infrastructures to be tested and validated directly supports this objective. By facilitating distributed and repeatable testing, the developed probes can contribute to improving the reliability of industrial systems and supporting their evolution.

The project can also be linked to **SDG 12 – Responsible Consumption and Production**. The ability to automate and repeat certain testing operations makes it possible to identify problems more quickly after modifications to a system. Better validation of infrastructures can therefore help reduce failures, unnecessary interventions, and the resources required for maintenance.

Finally, the project has an indirect connection with **SDG 8 – Decent Work and Economic Growth**. Automating certain testing operations reduces the amount of manual and repetitive work performed by engineers and operators. This allows them to dedicate more time to analysing results, solving problems, and improving systems.

Overall, the project contributes primarily to SDG 9 by providing an innovative solution for improving the validation and reliability of OT infrastructures. It also indirectly supports SDGs 12 and 8 by promoting more efficient use of resources and reducing repetitive manual testing operations.

3 | Analysis

This section presents the analysis conducted to identify a suitable hardware platform for the development of the probe.

The analysis first defines the hardware requirements derived from the overall architecture of the probe. Several hardware platforms were then investigated and compared based on these requirements. The NanoPi R5S, the Odroid H4 combined with a Netcard 3, and the Raspberry Pi devices available at the school were considered as potential solutions. The section concludes with the selection of the hardware platform used for the implementation of the probe.

Contents

3.1 Hardware requirements	7
3.2 NanoPi R5S	7
3.3 Odroid H4 + Netcard 3	8
3.4 Raspberry Pi 4	9
3.5 Hardware selection	9

3.1 Hardware requirements

As part of this project, several hardware constraints were defined to ensure the proper operation of the probe. The main requirements included the presence of at least **three Ethernet ports**, as well as compatibility with a **Linux** operating system to enable the execution of applications developed in Go.

More specifically, the probe requires three network interfaces to fulfil its different roles. Two Ethernet interfaces are dedicated to the test network. Having two separate interfaces is necessary to allow the probe to operate as a network bridge: the probe can be placed between two network devices, with each device connected to one of the two interfaces. The two interfaces can then be bridged so that traffic can be forwarded transparently from one side to the other while being monitored by the probe. The third interface is dedicated to the coordination network and is used for communication with the coordinator through the MQTT broker. This dedicated interface keeps the coordination traffic separate from the traffic exchanged through the test interfaces.

Based on these requirements, three hardware solutions were considered: the NanoPi R5S, the Odroid H4 combined with a Netcard 3, and the Raspberry Pi devices available at the school. These Raspberry Pi is limited to two Ethernet ports, but an additional network interface can be achieved by using Wi-Fi to connect to the manager.

3.2 NanoPi R5S

The first potential hardware platform considered for the probe was the NanoPi R5S.



Figure 1: Nano Pi R5S⁵

The NanoPi R5S[1] is a compact open-source IoT gateway developed by FriendlyElec, designed for networking applications. It is powered by a Rockchip RK3568B2 processor and includes 2GB or 4GB of RAM, along with eMMC storage, providing sufficient resources for embedded network tasks.

In the context of this project, the NanoPi R5S meets the key hardware requirements. It features three Ethernet ports (two 2.5GbE and one 1GbE), which directly supports the intended architecture: two interfaces can be used for network interconnection and traffic

⁵NanoPi R5S image[1]

capture, while the third is dedicated to communication with the manager. This eliminates the need for additional network hardware and simplifies deployment.

The board supports several Linux-based operating systems, ensuring compatibility with applications developed in Go. It can operate in headless mode, which is particularly suitable for a probe deployed in a network environment without direct user interaction.

3.3 Odroid H4 + Netcard 3

Another option considered was to use an Odroid H4 combined with a Netcard 3.



Figure 2: Odroid H4⁶



Figure 3: Netcard 3⁷

The ODROID-H4[2] is a compact x86 single-board computer developed by Hardkernel, designed for network, edge computing, and lightweight server applications. It is powered by an Intel processor N97 and supports DDR5 RAM, offering significantly higher general purpose computing performance compared to ARM-based alternatives. In this project, it is paired with the Netcard 3 expansion board[3], which provides additional high-speed network interfaces.

In the context of this project, the ODROID-H4 combined with the Netcard 3 meets the required hardware constraints. The system offers multiple Ethernet interfaces which makes it well-suited for a multi-interface architecture where traffic capture, network interconnection, and management communication must be logically separated. The ODROID-H4 runs standard x86 Linux distributions, ensuring full compatibility with Gobased applications and common networking tools. Like the NanoPi R5S, it can operate in headless mode, making it suitable for deployment as a network probe without direct user interaction.

⁶Odroid H4 image[2]

⁷Odroid H4 image[3]

3.4 Raspberry Pi 4

The third option investigated was to use the Raspberry Pi devices available at the school.

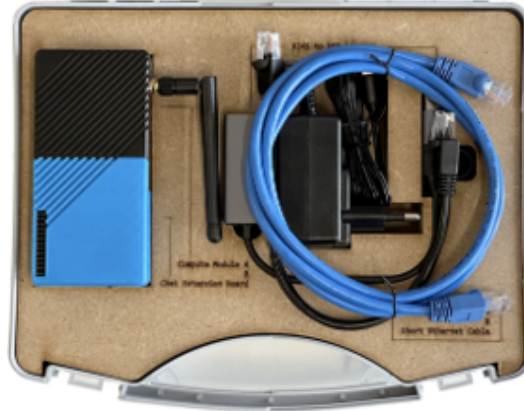


Figure 4: The CNet box (Raspberry Pi 4)⁸

The third option considered is the use of a Raspberry Pi 4. These devices are available at the school and have already been used in other courses, making them a convenient and accessible platform for development and testing. However, the Raspberry Pi 4 does not fully meet the project requirements, as it only provides two Ethernet interface. To compensate for this limitation, Wi-Fi can be used as an additional network connection to communicate with the manager while Ethernet is used for the other network roles.

3.5 Hardware selection

The NanoPi R5S was selected as the hardware platform for the probe. It provides the required three Ethernet interfaces directly, allowing the proposed network architecture to be implemented without any additional network hardware. Two interfaces can be dedicated to the test network, while the third can be used for the coordination network.

The NanoPi R5S also provides sufficient computing resources for the probe's workload, including network testing, packet capture, and the execution of the Go application. Its Linux compatibility allows all the required networking tools and libraries to be used, while its compact form factor and headless operation are well suited to deployment as a dedicated network probe.

Although the ODROID-H4 with the NetCard 3 provides greater computing performance and additional network interfaces, these capabilities are not required for this project and come with additional cost and hardware complexity. The Raspberry Pi 4 is readily available and suitable for development, but its two Ethernet interfaces do not directly satisfy the required architecture and would require Wi-Fi to provide the coordination interface.

The NanoPi R5S therefore provides the best balance between network interfaces, performance, simplicity, cost, and suitability for the intended deployment.

⁸The image of the CNet box comes from a CNet laboratory

4 | Design

This chapter presents the design of the probe and the system in which it is intended to operate. It first describes the two network environments considered for the deployment of the probe, as well as the requirements that the probe must fulfil. The software architecture of the probe is then presented, followed by a description of its workflow. The main libraries used to implement the proposed architecture are subsequently introduced. Finally, the communication interface between the probe and the coordinator is described, along with the deployment architecture used to run the probe.

Contents

4.1 Overview of the dual-environment system	11
4.2 Requirements Specification	12
4.3 Architecture of the probe	14
4.4 Libraries used	16
4.5 Interface with the coordinator	17
4.5.1 probes/announce	17
4.5.2 probes/discover	18
4.5.3 probes/{probe_id}/control	19
4.5.4 probes/{probe_id}/command	20
4.5.5 probes/{probe_id}/response	20
4.6 Deployment	21

4.1 Overview of the dual-environment system

The system designed for the deployment of the probes is divided into two separate environments: a **configuration environment** and a **test environment**. These environments are completely isolated from each other, and each has its own coordinator and MQTT broker. This separation makes it possible to prepare and configure the probes before deploying them in the OT network where the DUT (Device Under Test) will be tested.

The following diagram illustrates the overall architecture of the configuration and test environments, including the probes, MQTT brokers, coordinators, and the different networks involved.

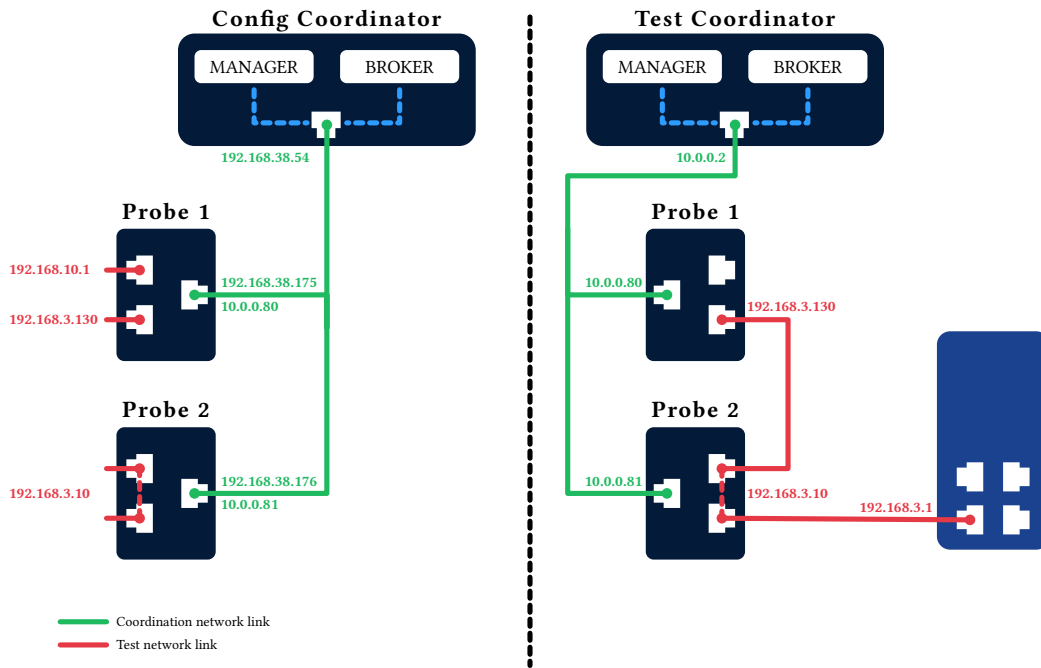


Figure 5: Diagram of the dual-environment system

The **configuration environment** is a permanent and controlled environment with a DHCP server and access to the Internet. It is used to **prepare and configure** the probes before their deployment in a test environment. Its MQTT broker is also permanent, meaning that its address and configuration are normally unchanged and always known to the probes. The information required to connect to this MQTT broker is defined in a **.env** file. This environment provides the probe with a reliable network from which it can initially connect to the coordinator and receive the parameters required for deployment. These parameters can include the static IP addresses and MQTT credentials needed to operate in the target test environment. This allows the probe to be prepared before deployment in a network that may not provide DHCP, Internet access, or other network configuration services.

The **test environment** represents the actual OT network in which the probe will be deployed. Its configuration can vary significantly, and it may not provide services such as DHCP. Configuring the probe beforehand in a known and controlled environment is

therefore essential. Once deployed in the OT network, the probe can operate using its pre-configured parameters without requiring any changes to the existing network infrastructure. However, the probe's configuration can also be modified remotely while it is deployed in the test environment, allowing its network parameters and other configuration settings to be adapted to the specific requirements of the tests being performed.

The example shown in the diagram illustrates how a probe is prepared before being deployed in a test environment. Initially, the two probes are connected to the configuration environment, where a DHCP server assigns an IP address to their **eth0** interfaces (**192.168.38.175** and **192.168.38.176**). These addresses allow the probes to establish communication with the Config Coordinator and receive their configuration. One of the probes is configured to operate as a bridge between its test interfaces. The probes are then configured with their specific test environment parameters, it's include **static IP** addresse for the interfaces **eth0**, **eth1**, **eth2** and the **bridge**, as well as the **MQTT credentials** required to connect to the Test Broker. They therefore have both DHCP-assigned address and a static IP address on **eth0**, while connected in the configuration environment. Once configured, the probes are moved to the test environment and connected to the OT network. They use the network parameters configured during the **configuration phase**, including their static IP addresses and MQTT credentials, to establish communication with the **Test Broker**. The IP address previously assigned by DHCP in the configuration environment is no longer used, and only the pre-configured static IP address remains active on the coordination interface. The probes can therefore operate in the test environment without requiring additional configuration of the OT network, allowing them to receive commands from the Test Coordinator and execute the required tests on the DUT.

4.2 Requirements Specification

The probe must provide a set of core functionalities required for its integration into the environment. It must first be able to register with the coordinator and report the capabilities it supports. In addition, the probe must expose the configuration parameters that can be modified remotely and apply any configuration changes received from the coordinator. These configuration settings must be saved locally so that they are retained across probe restarts.

The probe must also support the execution of commands requested by the coordinator. It must receive commands, execute the corresponding tests, return their results, and maintain a local log of the operations performed.

To support the dynamic integration of capabilities, each command exposed by the probe must provide its complete definition. This definition contains both the **information required to invoke the command**, such as its name and execution parameters, and the **metadata required to represent it in the coordinator's web interface**. The same principle applies to the probe configuration. For each configurable parameter, the probe provides the information required by the coordinator to display and modify it. This approach allows the coordinator to **dynamically** construct its user interface based on the information provided by the probe. As a result, new commands or configuration parame-

ters can be integrated without modifying the coordinator's underlying logic, **simplifying** the addition and integration of new probe functionalities.

The probe must also provide flexible network configuration capabilities. The two interfaces dedicated to the test network (**eth1** and **eth2**) must be capable of operating as a **bridge**. This allows the probe to be placed between two network devices and transparently forward the traffic between them while capturing it, without interrupting the connection.

All probe network interfaces must support both **static and dynamic IP** address configuration. This allows the probe to adapt to the different network configurations encountered in the test environments.

Finally, the architecture based on two separate environments introduces additional network requirements. Since the configuration process differs between the configuration and test environments, the probe must be able to determine which MQTT broker is reachable through its coordination interface and dynamically apply the configuration associated with that environment.

4.3 Architecture of the probe

The probe software is organized into several packages, each responsible for a specific part of its functionality. This modular architecture separates the different responsibilities of the application while allowing the components to interact with each other. The following section presents the overall architecture of the probe, from its main components to the internal structure of the `probe` package.

The following diagram shows the main packages of the probe and the relationships between them. It illustrates how the probe package interacts with the supporting packages to provide the required functionality.

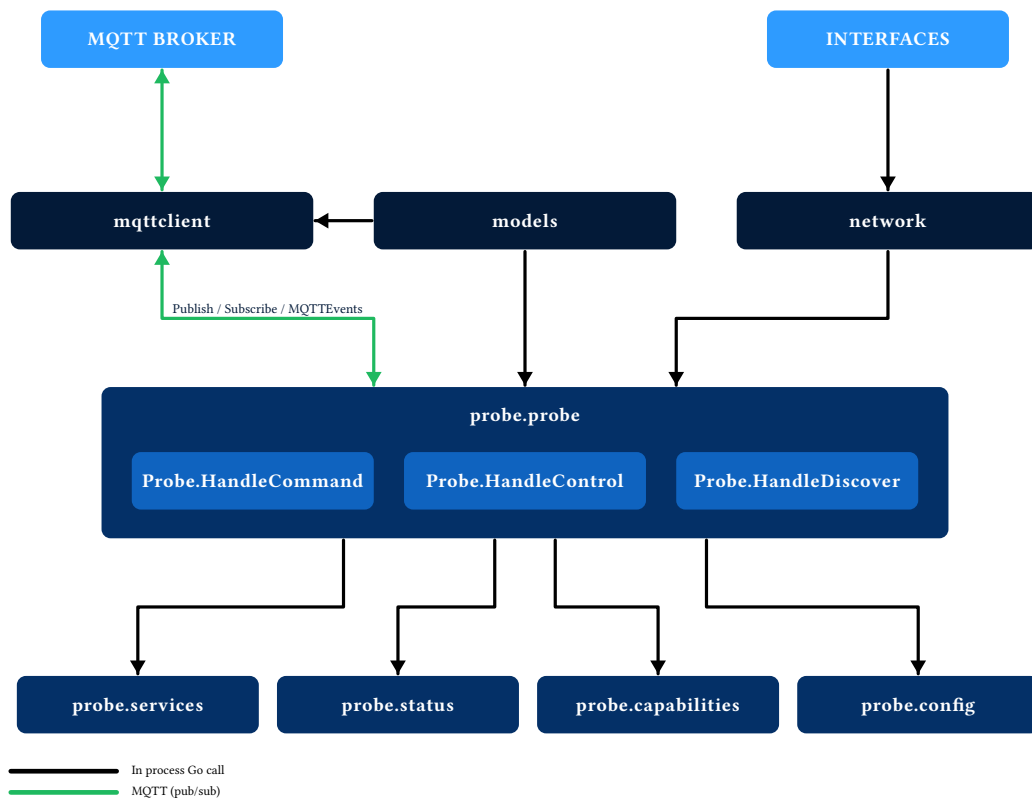


Figure 6: Architecture of the probe

The architecture shown in the diagram is composed of four main packages: `mqttclient`, `network`, `models`, and `probe`. The `probe` package contains the main execution logic of the application and is further divided into several sub-packages. These sub-packages are `probe.probe`, `probe.services`, `probe.status`, `probe.capabilities` and `probe.config`. Therefore, these five components are part of the probe package and are shown to illustrate its internal structure and the relationships between its components.

The main responsibilities of the four main packages are summarized in the following table.

Package	Responsability
mqttclient	This package implements the MQTT client used to communicate with the MQTT broker. It provides functions to create, configure and reconfigure the client, connect to a broker, subscribe or resubscribe to topics, and publish messages. It also generates events when the client connects, disconnects, or fails to connect to a broker.
network	This package handles the probe's network interfaces. It provides functions to retrieve and modify IP addresses, create network bridges, and perform other network-related operations required by the probe.
models	This package defines the data structures used for all MQTT messages exchanged by the probe. These structures ensure that messages are consistently formatted throughout the application.
probe	The probe package contains the main execution logic and relies on the <code>mqttclient</code> , <code>network</code> , and <code>models</code> packages to perform its operations.

Table 1: Responsibility of the main packages of the probe

The probe package is further divided into several **sub-packages**, each dedicated to a specific functionality of the probe. This organization separates the different responsibilities of the application and makes the code easier to maintain and extend. The following table presents these sub-packages and their respective responsibility.

Package	Responsability
probe	Contains the core logic of the probe, including its configuration, state, and main operations. It also defines the callback functions (<code>HandleCommand</code> , <code>HandleControl</code> and <code>HandleDiscover</code>) used to handle messages received on the MQTT topics to which the probe is subscribed.
services	Contains the implementations of the different command supported by the probe, such as ping, capture, portscan, HTTP request, TCP/UDP message and pcap execution.
status	Handles the periodic sending of status messages to report the current state of the probe.
capabilities	Contains the definitions of all commands supported by the probe. These definitions specify the parameters required to execute each action, as well as metadata used to display the commands in the coordinator's web interface.
config	Handles the probe configuration parameters as well as their persistence in a JSON configuration file.

Table 2: Responsibility of the internal packages of the probe package

The workflow of the probe is as follows. During startup, the probe is created and loads its configuration from the JSON configuration file (`probe.config` package). It then searches for an available MQTT broker on its **coordination network**, which can be either the configuration broker or the test broker. As soon as a broker is detected, the MQTT client is configured with the parameters corresponding to that broker, and a connection is established.

Once the connection is established, the probe **subscribes** to the required topics and associates a specific **callback function** to them. Each callback is responsible for processing the received message and triggering the corresponding action. This architecture allows the probe to process messages asynchronously.

When the connection to the broker and all topic subscriptions have been established, the probe announces itself to the coordinator by sending its **list of capabilities** and its **list of configuration parameters**. The capabilities and configuration parameters of this lists contains the **metadata** required by the coordinator to display and dynamically manage its interface. The probe then waits to receive messages from the coordinator. When a message is received, the callback associated with the corresponding topic is executed to process the message and trigger the appropriate action.

If the probe **loses its connection** to the MQTT broker, it enters a **reconnection** state and continuously searches for an available broker on its coordination network. When a broker is detected, the MQTT client is **reconfigured** with the parameters corresponding to that broker, and it **resubscribe** to the topics. This process allows the probe to switch between environments and automatically adapt its configuration to the broker it connects to. Once the connection has been re-established, the probe reannounce itself and waits for messages from the coordinator.

4.4 Libraries used

The implementation of the probe relies on several external libraries, each providing specific functionality. The following table presents these libraries, the packages that use them, and a brief description of their purpose.

Library	Package	Description
paho mqtt [4]	<code>mqttclient</code>	The paho mqtt library provides MQTT client functionalities for connecting to a MQTT broker, subscribing to topics and publishing messages.
netlink [5]	<code>network</code>	The netlink library provides a simple netlink library for go. It give access to features such as managing interfaces, IP addresses, and network bridges.

Table 3: Libraries used in the probe

The libraries used for the implementation of each probe command, such as ping, packet capture, or port scanning, are mentioned in the dedicated sections of the implementation chapter.

4.5 Interface with the coordinator

The communication between the coordinator and the network probes is entirely based on **MQTT topics**. These topics define the interface of the system, allowing both **command delivery** and **result collection** in a structured and scalable way.

The topic hierarchy follows a consistent naming convention, where each probe is identified by a unique identifier. This ensures that messages are properly routed to the intended recipient without ambiguity.

The topics hierarchy used in the system is as follows:

- **probes/announce**
- **probes/discover**
- **probes/{probe_id}/status**
- **probes/{probe_id}/control**
- **probes/{probe_id}/command**
- **probes/{probe_id}/response**

The following subsections describe each MQTT topic and its role within the system. All messages exchanged through these topics use the **JSON format** and contains a **timestamp** indicating the time of sending. For each topic, an explanation of the message structure is provided along with a real example.

4.5.1 probes/announce

This topic is used by all probes to **announce** their presence to the coordinator. The **announcement messages** allow the coordinator to identify available probes, retrieve their current configuration, and discover the actions they can perform.

A probe publishes an **announcement message** when it starts up, in response to a discovery request received on the **probes/discover** topic, and after any configuration update performed through the **probes/{probe_id}/control** topic. This ensures that the coordinator always has an up-to-date view of each probe.

The **announcement message** contains general information about the probe, including its identifier, firmware version, network information (IP addresses and MAC addresses of all available interfaces), supported capabilities with their parameters, the configuration groups and the current configuration values.

Here is a excerpt from a **announcement message** :

```

1 {
2   "probe_id": "probe-05e4fa",
3   "request_id": null,
4   "type": ["ethernet"],
5   "firmware_version" : "1.0.0",
6   "capabilities" : [
7     {
8       "action" : "ping",
9       "label" : "Ping",

```

JSON

```

10     "icon" : "network_ping",
11     "description" : "ICMP echo request test.",
12     "params" : [
13         {"key" : "interface", "label" : "Interface", "type" : "string", "example" :
14           "eth1"},
15         {"key" : "target", "label" : "Target", "type" : "string", "example" :
16           "google.ch"},
17         {"key" : "count", "label" : "Packet count", "type" : "number", "example" : "4",
18           "min" : 4, "max" : 100},
19         {"key" : "timeout_ms", "label" : "Timeout", "type" : "number", "example" :
20           "1000", "min" : 100, "max" : 30000}
21     ],
22     "result": [
23         {"key": "packets_sent"},
24         {"key": "packets_received"},
25         {"key": "packet_loss_percent"},
26         {"key": "rtt_min_ms"},
27         {"key": "rtt_avg_ms"},
28         {"key": "rtt_max_ms"}
29     ]
30 },
31 {
32     ...
33 }
34 ],
35 "config_groups": [
36     {"key": "network", "label": "Network", "icon": "lan"},
37     {"key": "mqtt", "label": "MQTT credentials", "icon": "hub"},
38     ...
39 ],
40 "config" : [
41     {"key" : "name", "label" : "Name", "type" : "string", "value" : "Probe LAN"},
42     {"key" : "status_interval_s", "label" : "Status interval", "type" : "number",
43       "value" : "1"},
44     ...
45 ],
46 "network": {
47     "eth0": {
48         "ip": [
49             "192.168.38.183/22",
50             "10.0.0.82/24"
51         ],
52         "mac": "fe:b6:53:86:ba:87"
53     },
54     ...
55 },
56 }

```

4.5.2 probes/discover

This topic is used by the coordinator to **request** all connected probes to **announce** themselves again. When a probe receives a message on this topic, it responds by publishing an **announcement message** on the `probes/announce` topic.

The **discovery message** contains only a *request_id* field, which uniquely identifies the request. This identifier is included in the corresponding **announcement message**, allowing the coordinator to associate each response with the original discovery request.

4.5.2.1 probes/{probe_id}/status

This topic is used by each probe to **periodically** inform the coordinator that it is still online. The publication interval is configurable through the probe configuration parameters.

The **status message** contains some information about the probe, including its name, state (online or disabled), uptime, CPU and memory usage, as well as temperature. In case of unexpected disconnection, the MQTT LWT (Last Will Testament) mechanism is used to automatically publish an offline **status message**, allowing the coordinator to be informed about a probe disconnection

Here is an **status message** example :

```
1 {
2   "name": "Probe LAN",
3   "state": "online",
4   "uptime_seconds": 4128,
5   "cpu_percent": 0.5050505050473733,
6   "memory_percent": 4.2865128805403625,
7   "timestamp": 1781256880393,
8 }
```

The offline **status message** send by the MQTT LWT only contains the field state, whose value is set to offline.

4.5.3 probes/{probe_id}/control

This topic is used by the coordinator to **configure** a specific probe.

The **configuration message** contains a unique ID to distinguish each configuration request and a list of parameters with the value to be applied. The available configuration are defined by the name contained in the *key* field from the configuration list provided by the probe in its **announcement message**.

Here is a excerpt from a **configuration message** :

```
1 {
2   "id": "setup-1783320613529525977",
3   "params": {
4     "name": "Probe LAN",
5     "status_interval_s": 30,
6     "enabled": true,
7     "bridge_mode": false,
8     ...
9   }
10 }
```

4.5.4 probes/{probe_id}/command

This topic is used by the coordinator to send a **command** request to a specific probe.

The **command message** contains a unique ID to distinguish each command request, the name of the action to execute and a list of parameters required for its execution. The list of parameters for each action is defined in the *params* field of the corresponding capability description, provided in the *capabilities* field of the **announcement message**.

Here is an example of **command message** :

```
1 {
2   "id": "2d5de829-ccdb-404d-91f8-be0835a795b2",
3   "action": "ping",
4   "params": {
5     "interface": "eth0",
6     "target": "192.168.3.10",
7     "count": 4,
8     "timeout": 1000
9   }
10 }
```



4.5.5 probes/{probe_id}/response

This topic is used by a specific probe to **answer** a configuration request (`probes/{probe_id}/control`) or a command request (`probes/{probe_id}/command`).

The **response message** contains the identifier of the request to which it corresponds, a status field (success, error, processing), an error that follows a generic format composed of an error code and an error message, and a result field. When the response message is associated with a **configuration request**, the *result* field is always empty. When it's sent in response to a **command request**, the format of the *result* field depends on the executed action. The list of results expected for each action is defined in the *result* field of the corresponding capability description, provided in the *capabilities* field of the **announcement message**.

Here is an example of **response message** with no errors :

```
1 {
2   "id": "2d5de829-ccdb-404d-91f8-be0835a795b2",
3   "status": "success",
4   "error": {
5     "code": 0,
6     "message": ""
7   },
8   "result": {
9     "packets_sent": 4,
10    "packets_received": 4,
11    "packet_loss_percent": 0,
12    "rtt_min_ms": 1.063101,
13    "rtt_avg_ms": 1.157672,
14    "rtt_max_ms": 1.209514
15  },
16 }
```



4.6 Deployment

The probe software is deployed as a Docker container on the NanoPi R5S. Docker allows the application and its required dependencies to be packaged together, making the deployment process consistent between different probes.

The **Dockerfile** defines how the probe application is built and packaged into a Docker image. It uses a **multi-stage build** with three stages. The `deps` stage provides the Go environment and installs the dependencies required to compile the application. The second stage, named `build`, inherits the environment from the `deps` stage. The source code is copied into the container and the probe is compiled into a single executable binary named `probeBin`. Finally, the `probe` stage uses a lightweight Debian image (`debian:bookworm-slim`) and installs only the dependencies required by the probe. The compiled binary is then copied from the `build` stage into the final image and configured as the container's entry point. This approach keeps the final image lightweight by excluding the Go compiler and other build dependencies.

The Docker image is published to the **GitHub registry** (ghcr.io) and can then be pulled by the NanoPi during deployment. Docker Compose is used to start and configure the probe container. The deployment also requires a `.env` file containing the configuration parameters specific to the environment. These parameters are documented in the `.env.example` file and include the MQTT credentials, as well as the path to the JSON file used to store the probe configuration. This `.env` file is provided separately from the Docker image.

The Docker Compose configuration defines how the container is started and provides the required network configuration, environment variables, and permissions. In particular, the container uses the host network mode (`network_mode: host`) to access the NanoPi's network interfaces directly, which is required for the probe's network operations and bridge configuration. The container is also granted the necessary privileges (`privileged: true`) to perform these low-level network operations.

The deployment process consists of building the Docker image for the ARM64 architecture using `docker compose build`, publishing it to the container registry using `docker compose push`. On each NanoPi, the image is then retrieved with `docker compose pull` and started using `docker compose up -d`, together with the appropriate `.env` file. This approach allows the same Docker image to be reused on multiple probes while keeping environment-specific configuration separate from the application.

The `Dockerfile`, `docker-compose.yml`, and `.env.example` file used for the deployment are provided in the appendices of this report.

5 | Implementation

This section presents the implementation of the probe and details the main components developed during the project. It first describes the implementation of the probe's overall workflow and its connection to the broker, followed by the communication with the coordinator and the definition of the formats used for configurations and capabilities, including the metadata required for their display in the coordinator web interface. The configuration of the network interfaces is then presented, followed by a detailed description of the different capabilities implemented on the probe.

Contents

5.1 Implementation of the probe workflow	23
5.2 Broker detection and connection	24
5.3 Implementation of the communication with the coordinator	24
5.3.1 Discovery request and status	25
5.3.2 Configuration request	26
5.3.3 Command request	27
5.4 Implementation of the configuration parameters list	28
5.4.1 Format of the configuration parameters groups	28
5.4.2 Format of the configuration parameters list	29
5.5 Implementation of the capabilities list	30
5.6 Configuration of the interfaces	32
5.6.1 Configuration of IP address	33
5.6.2 Creation of a bridge	34
5.6.3 Configuration of a DHCP server	35
5.6.4 Issue encountered	35
5.7 Implementation of the capabilities	36
5.7.1 Ping	36
5.7.2 Capture	37
5.7.3 Port scan	38
5.7.4 HTTP request	38
5.7.5 TCP request	39
5.7.6 TCP echo server	39
5.7.7 UDP request	40
5.7.8 UDP echo server	41
5.7.9 Send pcap	41

5.1 Implementation of the probe workflow

The probe workflow is implemented as a state machine composed of three states: **StateStarting**, **StateRunning**, and **StateConnecting**. Each state is responsible for a specific part of the probe's lifecycle, from its initial configuration to normal operation and connection recovery.

During **StateStarting**, the probe is created and loads its configuration from the JSON configuration file. It then searches for an available MQTT broker on the coordination network. Depending on the environment, this can be either the configuration broker or the test broker. Once an available broker is detected, the MQTT client is configured with the corresponding connection parameters and attempts to establish a connection. After the connection has been successfully established, the probe subscribes to the required MQTT topics and associates a callback function with each topic. Once the connection and all required subscriptions are successfully established, the probe transitions to **StateRunning**.

In **StateRunning**, the probe first announces itself to the coordinator by publishing an **announcement message** containing its capabilities and configurable parameters. It then starts periodically sending status messages and waits for incoming message from the coordinator. When a message is received, the callback associated with its topic processes the message and triggers the appropriate action. If the probe loses its connection to the MQTT broker, the MQTT client generates an **EventMQTTDisconnected** event. The probe stops its periodic status messages and transitions to **StateConnecting**.

The **StateConnecting** is responsible for recovering the MQTT connection. It continuously searches for an available broker on its coordination network. When a broker becomes available, the MQTT client is reconfigured with the corresponding parameters and attempts to establish a new connection. If the connection succeeds, the client generates an **EventMQTTConnected** event. The probe then restores its subscriptions and transitions back to **StateRunning**. If the connection attempt fails, the client generates an **EventMQTTConnectionFailed** event, and the probe remains in **StateConnecting** and continues searching for an available broker.

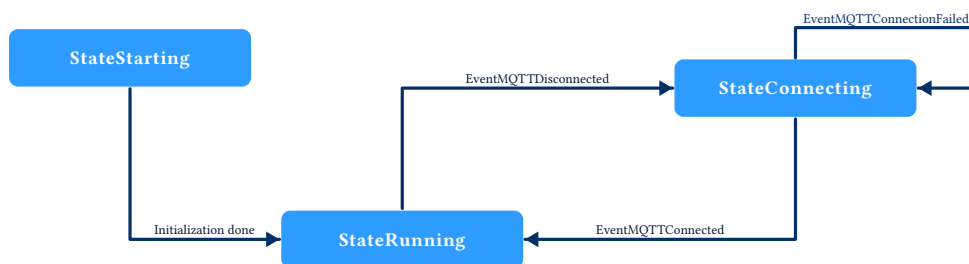


Figure 7: State machine that implement the probe workflow

5.2 Broker detection and connection

As explained, the probe is designed to operate in two different environments: the **configuration environment** and the **test environment**. Each environment has its own MQTT broker. The probe must therefore have two broker configurations, each allowing it to establish a connection with one of the two brokers:

- **BrokerConfigParams** : configuration of the broker used by the configuration environment
- **TestBrokerParams** : configuration of the broker used by the test environment

The **BrokerConfigParams** parameters are defined in the `.env` file and loaded when the probe starts. They cannot be modified while the program is running: any change to the `.env` file requires the probe to be restarted to take effect. The **TestBrokerParams** parameters, on the other hand, are part of the probe's internal configuration and can be modified while it is running when a configuration message is received on `probes/{probe_id}/control`.

Once both configurations are available, the probe must determine which broker is reachable from its **coordination network** (eth0). To do so, it attempts to establish a **TCP connection** with each configured broker. When a connection is successfully established with one of them, the probe identifies the broker available on its network and can therefore determine which environment it is operating in. This information is important because the configuration of the network interfaces of the probe, differs between the two environments.

Once the broker has been identified, the MQTT client is configured with the corresponding parameters and it connects to the available broker. After the connection is established, the MQTT client subscribes to the different topics required for its operation.

This mechanism is also used when the MQTT connection is lost (**StateConnecting**). Whenever a **disconnection** occurs, the probe searches again for the broker accessible from its network. As soon as a TCP connection is established with one of the brokers, that broker is identified and the probe MQTT client is **reconfigured** with the corresponding parameters. The probe can then attempt to establish a new MQTT connection and resubscribe to the required topics.

This approach allows the probe to automatically adapt to a change of environment without requiring a restart.

5.3 Implementation of the communication with the coordinator

During the initialization of the probe (**StateStarting**), a map is created to associate each subscribed MQTT topic with its corresponding callback function. This allows the MQTT client to automatically call the appropriate function when a message is received. Each type of message is therefore handled by a dedicated callback, keeping the logic associated with each topic separate.

Three callback functions were defined:

- **handleDiscover** (for the message received on `probes/discover`)
- **handleControl** (for the message received on `probes/{probe_id}/control`)
- **handleCommand** (for the message received on `probes/{probe_id}/command`)

These callback functions are implemented in the `probe.probe` package. When a message is received on one of the topics, the MQTT client automatically calls the associated callback function and passes the message payload as a parameter. The function can then process the message and perform the corresponding operations.

The following subsections illustrate, using sequence diagrams, how the communication with the coordinator takes place.

5.3.1 Discovery request and status

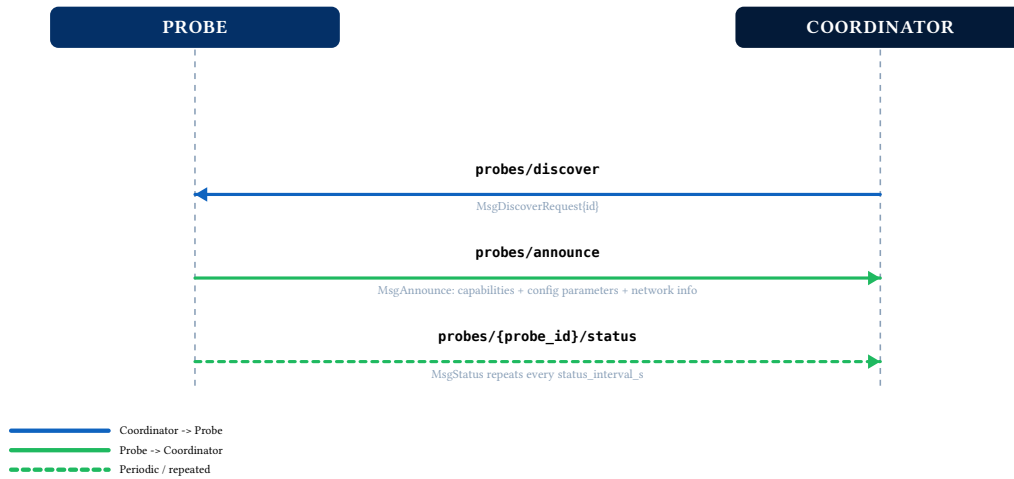


Figure 8: Sequence diagram - Discovery request and status heartbeat

When the probe receives a discovery message on the `probes/discover` topic, it announces its presence again by sending an **announcement message** on the `probes/announce` topic.

As soon as a message is received on `probes/discover`, the **handleDiscover** callback function is executed. Its sole purpose is to send the **announcement message** using the `request_id` received in the discovery message.

The probe also periodically publishes **status messages** (heartbeat mechanism). To send these messages periodically, a dedicated function is started in a **goroutine** running alongside the rest of the system. This function is implemented in the `probe.status` package and is responsible only for sending **status messages**. The goroutine is stopped when the connection to the broker is lost and restarted when the connection is re-established. A channel is used to notify the goroutine when the probe configuration is updated (`probes/{probe_id}/control`). This allows the goroutine to receive the updated configuration of the probe and adjust the interval between status messages accordingly.

5.3.2 Configuration request

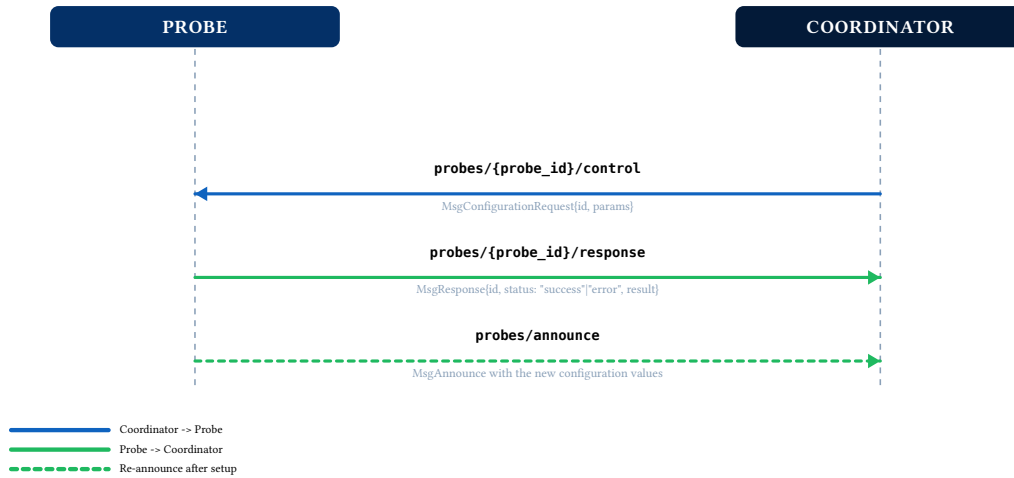


Figure 9: Sequence diagram - Control request

When the probe receives a **configuration message** on the `probes/{probe_id}/control` topic, it must update its settings and send a **response message** on the `probes/{probe_id}/response` topic. The response can either be a **success response** if the probe was updated successfully, or an **error response** if the update failed.

As soon as a message is received on `probes/{probe_id}/control`, the `handleControl` callback function is executed. This function first checks whether the received parameters are valid and consistent. If they are, it updates the probe's configuration and saves the new parameters in the configuration JSON file. This file provides persistent storage for the probe's configuration, allowing the settings to be restored after a restart.

After each configuration change, the probe sends an **announcement message** on the `probes/announce` topic to provide the coordinator with an up-to-date view of the probe's configuration.

5.3.3 Command request

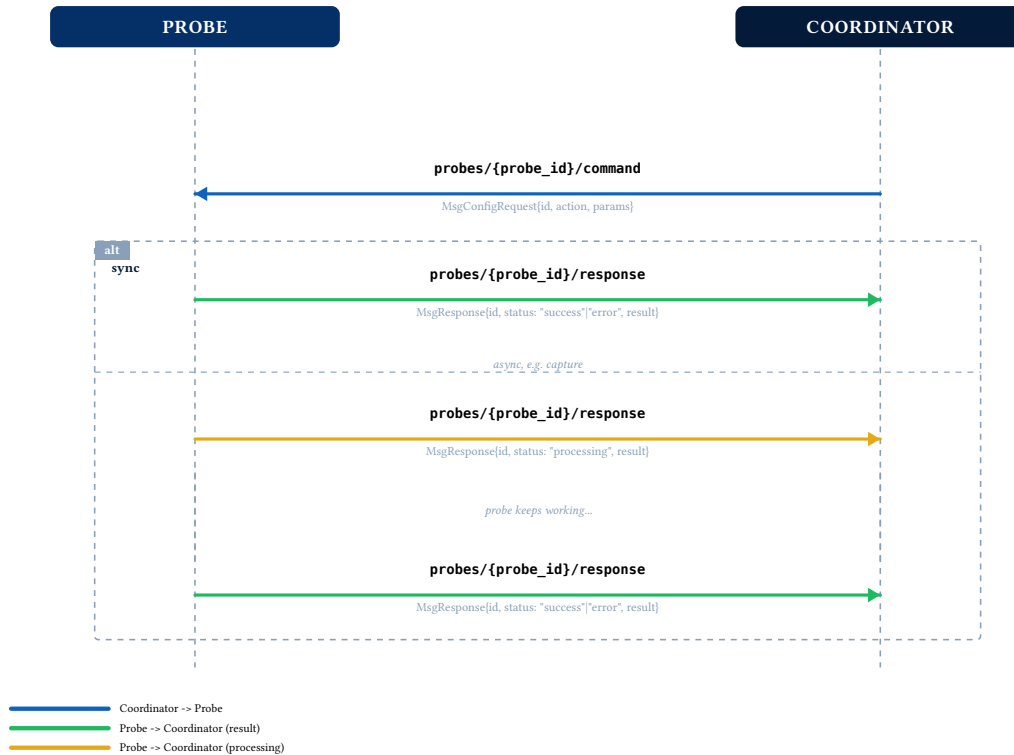


Figure 10: Sequence diagram - Command request

When the probe receives a **command message** on the `probes/{probe_id}/command` topic, it must execute the requested command and send its result in a **response message** on the `probes/{probe_id}/response` topic.

For a command, three types of response messages can be sent:

- **Success response:** sent if no error occurs during the validation or execution of the command. Contains the results obtained after the command has been executed.
- **Error response:** sent if an error occurs during the validation or execution of the command. Contains the error composed of its error code and message
- **Processing response:** sent when a command requires a longer execution time or involves operations that need to run in parallel, such as a capture or a port scan. This message informs the coordinator that the command has started and allows it to continue processing other operations without waiting for the final result. The final response is sent once the command has completed.

During the probe's initialization (**StateStarting**), a map of executors is created, associating each command name with its corresponding executor function (for example, "ping" → `pingExecute()`). These functions are implemented in the `probe.services` package, and their operation is described later in this report.

When a message is received on the `probes/{probe_id}/command` topic, the `handleCommand` function is executed. This function first checks whether the received parameters are valid. If they are, it uses the `action` field from the command message to find the corresponding

executor in the executor map. The selected executor function is then executed in a goroutine, allowing multiple commands to run in parallel without blocking the rest of the probe.

5.4 Implementation of the configuration parameters list

This section of the implementation explain how the list of configuration parameters are defined before being sent to the coordinator through the announcement message (`probes/announce`). As explained previously, each configuration parameter is defined with all the metadatas required for the coordinator's web interface to display it.

To achieve this, a generic configuration parameter format containing all the necessary information was defined. The structure of this message is located in the `models` package and is defined by the `Param` structure.

Some configuration parameters are grouped together in a parameter group. This is, for example, the case for the parameters related to the MQTT credentials or to a specific network interface. These groups also needs to contain some metadata to enable the coordinator to correctly display them on the web interface, so a format was also defined and its defined by the `Group` structure.

The following sections first describe how these parameter groups are defined, then it explain the definition of the configuration parameters.

5.4.1 Format of the configuration parameters groups

The list of configuration groups currently implemented in the system are defined below:

```

1 {
2   "config_groups": [
3     {"key": "network", "label": "Network", "icon": "lan"},
4     {"key": "mqtt", "label": "MQTT credentials", "icon": "hub"},
5     {"key": "bridge", "label": "Bridge", "icon": "device_hub",
6       "visible_if": {"key": "bridge_mode", "value": true}
7     },
8     {"key": "eth1", "label": "Interface eth1", "icon": "settings_ethernet",
9       "visible_if": {"key": "bridge_mode", "value": false}
10    },
11    {"key": "eth2", "label": "Interface eth2", "icon": "settings_ethernet",
12      "visible_if": {"key": "bridge_mode", "value": false}
13    },
14  ],
15 }
```

The Network group contains the configuration parameters of the coordination interface (eth0) of the probe, while MQTT credentials contains the information required to connect with the MQTT broker of the test environment. The Bridge group contains the parameters related to the bridge configuration and is displayed when bridge mode is enabled. When bridge mode is disabled, the interfaces eth1 and eth2 groups are displayed instead, allowing each test interface to be configured individually.

The different fields of a configuration group (`Group` struct) are described below. The fields shown in bold are mandatory for all the configuration parameters groups.

Field	Description
key	The name used to identify the group in the configuration parameters.
label	The name of the group displayed in the web interface.
icon	The icon associated with the group. The icon name can be any Google Material Icon name, written in lowercase with spaces replaced by underscores.
visible_if	Defines a display condition. The group is displayed only when this condition is satisfied.

Table 4: Fields description of a configuration group format

The *visible_if* field contains two sub-fields, *key* and *value*. The key specifies the name of a probe configuration parameter, while value specifies the value that this parameter must have for the group to be displayed in the web interface. For example, in the configuration groups presented above, the bridge group, including all of its associated parameters, is only displayed when the *bridge_mode* parameter is set to true. This mechanism allows the web interface to display only the configuration parameters that are relevant to the current probe configuration, keeping the interface more concise and easier to use.

The list of the configuration groups is defined by a function of the `probe.probe` package. This function is called `SetConfigGroupsList` and it returns a `[]models.Group`. The probe includes this list in every **announcement message**. Adding a new configuration group to the **announcement message** requires to add new entry in the list implemented by this function.

5.4.2 Format of the configuration parameters list

The following is an excerpt from the list of configuration parameters provided by the probe:

```

1 {
2   "config" : [
3     {"key" : "name", "label" : "Name", "type" : "string", "value" : "Probe LAN"},
4     {"key" : "status_interval_s", "label" : "Status interval", "type" : "string",
5      "value" : "1"},
6     {"key" : "enable", "label" : "Enabled", "type" : "bool", "value" : "true"},
7     {"key" : "bridge_mode", "label" : "Bridge activated", "type" : "bool", "value" :
8      "false"},
9     {"key" : "ip", "label" : "IP address", "type" : "string", "value" : "10.0.0.80",
10    "group" : "network"},
11    ...
12  ],
13 }
    
```

The different fields of a configuration parameter (`Param` struct) are described below:

Field	Description
key	The name used to identify the configuration parameter. This key is used when sending the configuration parameter through the <code>probes/{probe_id}/control</code> message.
label	The name of the configuration parameter displayed in the web interface.
type	The type of value that can be provided for the parameter.
value	The actual value assigned to the configuration parameter.
group	The name of the configuration group to which the parameter belongs. This field is optional.

Table 5: Fields description of a configuration parameter format

The list of the configuration parameters is defined by a function of the `probe.probe` package. This function is called `SetConfigParamsList` and it returns a `[]models.Param`. The probe includes this list in every **announcement message**. Adding a new configuration parameter to the **announcement message** requires to add new entry in the list implemented by this function.

5.5 Implementation of the capabilities list

This section of the implementation explain how the commands list is defined before being sent to the coordinator through the announcement message (`probes/announce`). As explained previously, each command must be fully defined by the probe. This includes not only the information required to execute the command, but also all the metadata required to display and configure the command in the coordinator's web interface.

To achieve this, a capability format containing all the information required by the coordinator to display a command was defined. The structure of this message is located in the `models` package and is defined by the `Capability` struct.

The following example shows the capability definition for the ping command:

```

1 {
2   "capabilities" :[
3     {
4       "action" : "ping",
5       "label" : "Ping",
6       "icon" : "network_ping",
7       "description" : "ICMP echo request test.",
8       "params" : [
9         {"key" : "interface", "label" : "Interface", "type" : "string", "example" :
10          "eth1"},
11         {"key" : "target", "label" : "Target", "type" : "string", "example" :
12          "google.ch"},
13         {"key" : "count", "label" : "Packet count", "type" : "number", "example" : "4",
14          "min" : 4, "max" : 100},

```

```

12     {"key": "timeout_ms", "label": "Timeout", "type": "number", "example":
13       "1000", "min": 100, "max": 30000}
14   ],
15   "result": [
16     {"key": "packets_sent"},
17     {"key": "packets_received"},
18     {"key": "packet_loss_percent"},
19     {"key": "rtt_min_ms"},
20     {"key": "rtt_avg_ms"},
21     {"key": "rtt_max_ms"}
22   ]
23   ...
24 ]
25 }

```

The different fields of this message are described below:

Field	Description
action	The name of the command that must be provided in the <i>action</i> field of the command message (<code>topic probes/{probe_id}/command</code>)
label	The name of the command displayed in the web interface.
icon	The icon representing the command in the web interface. The icon name can be any Google Material Icon name, written in lowercase with spaces replaced by underscores.
description	A short description of the command.
params	The list of parameters required to execute the command. Each parameter follows a specific format (<code>CapabilityParam</code> struct), which is described below.
result	The list of results expected after the command is executed. Each result follows a specific format (<code>Result</code> struct), which is described below.

Table 6: Fields description of the Capability format

The fields of the format for a command parameter (`CapabilityParam` struct) is described below. The fields shown in bold are mandatory for all parameters of every command:

Field	Description
key	The name of the parameter that must be provided in the <code>params</code> field of the command message.
label	The name of the parameter displayed in the web interface.
type	The type of value that must be provided for the parameter.
example	An example of a possible value for the parameter, displayed to the user in the web interface.
min	The minimum value allowed for the parameter. This can be used to define limits in the web interface.
max	The maximum value allowed for the parameter. This can be used to define limits in the web interface.
unit	The unit associated with the parameter.
options	The possible values for the parameter when it must be selected from a predefined list. For example, this is used by the http command to select the HTTP method (GET, PUT, ...)

Table 7: Fields description of the CapabilityParams format

The expected results format is defined using a single field, **key**. This field is mandatory and contains the name of the result sent by the probe in the response message (`probes/{probe_id}/response`).

The definitions of all command descriptions are located in the `probe.capabilities` package as variables implementing the format described above (`models.Capability`). These variables are then grouped into a variable of type `[]models.Capability`, called `capabilities.CapabilitiesList`. The probe stores this list in its configuration and includes it in every **announcement message**.

Adding a new command to the **announcement message** therefore only requires creating a variable implementing the `models.Capability` structure and adding it to the `capabilities.CapabilitiesList` list.

5.6 Configuration of the interfaces

This section describes the configuration of the probe's network interfaces. As explained previously in the requirements section, the interfaces must be configurable to meet the network requirements of the different environments in which the probe can be deployed.

As described earlier, the system consists of two separate environments: the **configuration environment** and the **test environment**. In the configuration environment, the coordination network provides the probe with an IP address through DHCP. However, the coordination network in the test environment may not provide a DHCP server. It is therefore necessary to be able to configure a static IP address for the probe's **eth0**

interface, allowing it to use an address belonging to the coordination network of the test environment (`10.0.0.0/24` of the dual environment diagram). The same requirement applies to the interfaces connected to the test network (**eth1** and **eth2**). Since this network may also operate without a DHCP server, the probe must be able to assign static IP addresses to these interfaces. This allows an IP address compatible with the test network of the test environment to be manually defined when deploying the probe.

The two interfaces connected to the test network must also be able to operate in bridge mode. In this configuration, **eth1** and **eth2** are combined into a single bridge interface (**br0**) allowing traffic to be transparently forwarded between the two interfaces.

The ability to run a DHCP server on one of the probe's interfaces was also implemented. This was not a major requirement for the intended operation of the system, but it proved useful for debugging and testing during the development of the probe.

The implementation of these different features is described in the following subsections. To implement these features, the `network` package was developed using the netlink library. All parameters required for network configuration are received through the configuration message (`probes/{probe_id}/control`) sent by the coordinator. These parameters are divided into different groups depending on the part of the network configuration they concern.

5.6.1 Configuration of IP address

It is necessary to distinguish between two types of IP configuration in this system:

- Coordination network
- Test network

The IP configuration for these two networks is implemented differently for several reasons, which will be explained in the following sections.

To configure an IP address on a test network interface (eth1, eth2, or bridge), the following parameters are required:

Parameter	Description
IP	The IP address to assign to the interface. It must be provided in IPv4 format (example: 10.0.0.80).
Mask	The subnet mask in dotted-decimal notation (example: 255.255.255.0).

Table 8: Configuration parameters for a test interface

The configuration of a test network interface is performed as follows. First, the interface is reset by removing all of its existing IP addresses. If the **IP parameter** is *null*, the DHCP client is enabled on the interface, which then waits to receive a dynamic IP address from a DHCP server. If the **IP parameter** is *not null*, the DHCP client is disabled and the specified IP address is assigned to the interface together with its configured subnet mask (**Mask parameter**). This mechanism allows the test network interfaces to be configured with either a static or a dynamic IP address.

In the current version of the probe, the gateway of the test interfaces cannot be configured. A future improvement could therefore be to allow gateway configuration on these interfaces to support networks requiring specific routing.

To configure the IP address of the coordination network interface (eth0), the following parameters are required:

Parameter	Description
IP	The IP address to assign to the interface. It must be provided in IPv4 format (example: 10.0.0.80).
Mask	The subnet mask in dotted-decimal notation (example: 255.255.255.0).
Gateway	The gateway of the coordination network.
Subnet	The subnet of the coordination network in the test environment. It must be provided in CIDR notation (example: 10.0.0.0/24).

Table 9: Configuration parameters for the coordination interface

The configuration of the coordination network interface is handled differently from the test network interfaces. First, all IP addresses belonging to the network specified by the **Subnet parameter** are removed from **eth0**. This partial reset ensures that only the IP addresses associated with the test environment's coordination network are removed. This is important when the probe is connected to the configuration environment, as the IP address (received by the DHCP server of the configuration environment) used to communicate with the configuration MQTT broker must not be removed when configuring **eth0**. After this partial reset, if the **IP parameter** is *null*, the DHCP client is enabled on eth0, allowing the interface to obtain an IP address dynamically. If the **IP parameter** is *not null*, the environment to which the probe is connected is first determined. If the probe is connected to the **configuration environment**, the specified IP address is assigned to eth0 and the gateway is configured according to the **Gateway parameter**. The DHCP client remains enabled in this case. If the probe is connected to the **test environment**, the specified IP address and gateway are configured and the DHCP client is disabled. This distinction is important because disabling the DHCP client in the **configuration environment** would cause the probe to lose the IP address used to communicate with the configuration broker, resulting in a loss of connectivity with the broker.

5.6.2 Creation of a bridge

To create a bridge, the *bridge_mode* configuration parameter of the probe must be enabled. When this parameter is enabled, the probe creates a bridge interface named **br0** and configures it in the same way as the other test network interfaces (**eth1** and **eth2**). The procedure for configuring a test network interfaces was explained in the previous subsection.

5.6.3 Configuration of a DHCP server

To configure a DHCP server on an interface, the following parameters are required:

parametre	Description
DHCP activated	Boolean value determining whether the DHCP server should be enabled or disabled.
DHCP start adress	The first IP address of the DHCP address range.
DHCP stop address	The last IP address of the DHCP address range.
DHCP gateway	The gateway provided to clients by the DHCP server.

Table 10: Configuration parameters for the DHCP server configuration

When required, the DHCP server is created using `dnsmasq` [6]. Only interfaces belonging to the test network (eth1, eth2 or br0) can have a DHCP server configured.

5.6.4 Issue encountered

When we configure a **static IP** address on an interface, the DHCP client is disabled. It implies that the OS no longer knows the DNS of the **coordination network**. As a consequence I was unable to update my probe from the test environnement (pull a new version of the image from `ghcr.io`), despite the fact that the environnement had internet. I tried several methods to solve this problem, but I was unable to resolve it. I tried modifying the informations in the OS's `/etc/resolv.conf` file, but modifying the OS information from Docker container is complex. Following a discussion with the professor responsible for the Bachelor thesis, we concluded that it was not necessary to continue trying to solve this behavior, considering that it was outside the scope of this thesis and the time planned for this thesis was coming to an end. So pulling a new versions is only possible in the **configuration environnement**, which always provides the probe with an IP address through DHCP and therefore also provides it with DNS information.

The absence of known DNS server in the OS's `/etc/resolv.conf` caused another problem. When the OS starts, Docker copies the OS's `/etc/resolv.conf` into its own `/etc/resolv.conf`. Since the DHCP client of `eth0` is enabled by default when the OS starts, the Docker container `/etc/resolv.conf` initially contains the DNS informations provided by the DHCP. This happens because Docker copies the file before the probe program starts and configures a static IP address on `eth0`. As a result, after the startup, the Docker container knows the DNS server and the probe can resolve hostnames, allowing it to ping `google.ch` for example. However, when the Docker container is stopped (**docker compose down**) and started again (**docker compose up**), Docker updates its `/etc/resolv.conf` by copying the OS's file again that doesn't contains the DNS. Therefore, after restarting the Docker container, the probe is no longer able to resolve hostnames such as `google.ch`. This situation caused inconsistent behavior in the probe, so I tried to find a solution. During these tests, I used `net.Resolver` to force my program to resolve hostnames using a custom DNS server. The tests with this approach were relatively conclusive. However, this does not solve the initial problem of the missing DNS information in the OS.

The way `net.Resolver` was used is explained in the section dedicated to the `net.Resolver` proof of concept.

5.7 Implementation of the capabilities

During this thesis, several capabilities were implemented in the probe to allow it to perform different types of tests. This section describes the implemented commands, their purpose, the parameters required to execute them, and the results they provide. All the commands are implemented in the `probe.services` packet.

Although each command performs a different operation, their execution follows a common process. When the execution function is called, the command parameters are first validated to ensure that they are correct and complete. If an error is detected during this validation, the probe sends an error response to the coordinator through the `probes/{probe_id}/response` topic. This response contains the error code and the corresponding error message.

If the parameters are valid, the execution of the command can begin. For commands that may take some time to complete (capture and port-scan), the probe first sends a processing message to the coordinator through the same topic to indicate that the command has started. Once the execution is completed, the probe creates a response containing the results obtained and publishes it to the `probes/{probe_id}/response` topic.

Each command is implemented using libraries suited to its specific functionality.

5.7.1 Ping

The **ping** command is used to verify network connectivity between the probe and the DUT (Device Under Test). It uses the ICMP (Internet Control Message Protocol) to send Echo Request messages to a target address and wait for the corresponding Echo Reply messages.

The ping command uses the `prometheus-community/pro-bing` library[7] to send ICMP echo requests and collect the corresponding results

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface on which the ping is performed.
target	IP address or hostname of the target to ping.
count	Number of ICMP requests to send.
timeout_ms	Maximum time to wait for a response.

Table 11: Ping execution parameters

The results of a **ping** executed successfully contain:

Results	Description
packets_sent	Number of ICMP packets sent.
packets_received	Number of ICMP packets received.
PacketLossPercent	Percentage of packets lost.
rtt_min_ms	Minimum round-trip time.
rtt_avg_ms	Average round-trip time.
rtt_max_ms	Maximum round-trip time.

Table 12: Ping results

These values make it possible to evaluate not only the reachability of the target, but also the quality of the network connection.

5.7.2 Capture

The **capture** command is used to record the traffic passing through a network interface for a defined period of time. The captured packets can then be analyzed to verify the behavior of the DUT, identify the protocols being used, or diagnose potential communication problems.

The capture command uses the `gopacket` library[8] and its `pcap` [9] and `pcapgo` [10] packages to capture network packets and handle PCAP files.

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface on which the capture is performed.
filter	BPF filter applied during the capture.
timeout_s	Duration of the capture in seconds.

Table 13: Capture execution parameters

The results of a **capture** executed successfully contain:

Results	Description
pcap_b64	Return a pcap file of the capture encoded in base64

Table 14: Capture results

Returning a PCAP file is useful because it provides a convenient way to store network captures, by decoding the **pcap_b64** data and saving it as a file, and to compare captures obtained from different tests. Furthermore, this format is compatible with analysis tools such as Wireshark, allowing BPF filters to be applied afterwards to isolate only the relevant traffic from a complete network capture.

5.7.3 Port scan

The **port-scan** command uses Nmap to identify the open ports on the DUT. It is used to determine which services are accessible on the device and to verify that only the expected ports are exposed on the network.

The port scan command uses the `Ullaakut/nmap` library[11] to perform network port scans. The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface used to perform the port scan.
target	IP address or hostname of the target to scan.
ports	Ports of the target to scan (example: 80,443,8000-8020,8080).
sv	Boolean value used to enable or disable the service detection.

Table 15: Portscan execution parameters

The results of a **port scan** executed successfully contain:

Results	Description
ports	List of the scanned ports, including their ID, protocol, state (open/closed), and detected service.

Table 16: Portscan results

5.7.4 HTTP request

The **http** command uses Go's standard `net/http` package[12] to send HTTP requests. The `encoding/base64` package[13] is used to decode Base64-encoded request data, while `unicode/utf8` package[14] is used to handle UTF-8 encoded data.

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface used to perform the http request.
method	HTTP method used for the request, such as GET, POST or PUT.
header	HTTP headers to include in the request.
url	URL of the target HTTP resource.
body	Content of the HTTP request body. The body can contain JSON data, plain text, or Base64-encoded file content.

Table 17: HTTP execution parameters

The results of a **http** command executed successfully contain:

Results	Description
status	HTTP status code returned by the server.
statusText	Text associated with the returned HTTP status code.
httpVersion	HTTP version used for the response.
headers	HTTP headers returned by the server.
content	Content of the HTTP response body.

Table 18: HTTP results

5.7.5 TCP request

The **tcp** command uses Go's standard `net` package[15] to establish TCP connections and exchange data.

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface used to perform the TCP request.
source-port	Source port used to establish the TCP connection.
destination-ip	IP address of the destination.
destination-port	Destination port to which the TCP connection is established.
data	Data sent to the destination.
waiting-response	Boolean value indicating whether the probe should wait for a response from the destination.

Table 19: TCP execution parameters

The source IP address used for the TCP request is the IP address assigned to the network interface specified by the *interface* parameter.

The results of a **tcp** command executed successfully contain:

Results	Description
response	Data received from the destination. This field is empty if waiting-response is disabled.

Table 20: TCP results

5.7.6 TCP echo server

The **tcp echo server** uses Go's standard `net` package[15] to create a TCP server that receives and sends back the received data. It was mainly implemented to test the TCP request command using two probes: one probe runs the TCP echo server, while the other sends TCP requests to it. The response received by the sending probe can then be compared with the original message to verify that the TCP communication and data exchange are working correctly.

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface on which the TCP echo server listens for incoming connections.
port	TCP port on which the echo server listens for incoming connections.
timeout_s	Duration of the TCP echo server in seconds.

Table 21: TCP echo server execution parameters

The source IP address used for the TCP echo server is the IP address assigned to the network interface specified by the *interface* parameter.

The results of a **tcp echo server** command executed successfully contain:

Results	Description
response	List of the data received from the clients and sent back by the TCP echo server.

Table 22: TCP echo server results

5.7.7 UDP request

The **udp** command uses Go's standard `net` [15] package to create UDP connections and send datagrams.

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface used to perform the UDP request.
source-port	Source port used to establish the UDP connection.
destination-ip	IP address of the destination.
destination-port	Destination port to which the UDP connection is established.
data	Data sent to the destination.
waiting-response	Boolean value indicating whether the probe should wait for a response from the destination.

Table 23: UDP execution parameters

The source IP address used for the UDP request is the IP address assigned to the network interface specified by the *interface* parameter.

The results of a UDP request executed without errors contain:

Results	Description
response	Data received from the destination. This field is empty if waiting-response is disabled.

Table 24: UDP results

5.7.8 UDP echo server

The **udp echo server** uses Go's standard `net` package[15] to create a UDP server that receives and sends back the received data. It was mainly implemented to test the UDP request command using two probes: one probe runs the UDP echo server, while the other sends UDP requests to it. The response received by the sending probe can then be compared with the original message to verify that the UDP communication and data exchange are working correctly.

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface on which the UDP echo server listens for incoming connections.
port	UDP port on which the echo server listens for incoming connections.
timeout_s	Duration of the UDP echo server in seconds.

Table 25: UDP echo server execution parameters

The source IP address used for the UDP echo server is the IP address assigned to the network interface specified by the *interface* parameter.

The results of a UDP echo server request executed without errors contain:

Results	Description
response	List of the data received from the client and sent back by the UDP echo server.

Table 26: UDP echo server results

5.7.9 Send pcap

The **pcap** command sends the packets contained in a PCAP file through a selected network interface of the probe. It can be used to reproduce predefined network traffic and test how the DUT reacts to a specific sequence of packets.

The PCAP sending command uses the `pcap` package[9] from the `gopacket` library[8] to send packets through a network interface. The `encoding/base64` package[13] is used to decode the PCAP file received from the coordinator.

The parameters that must be provided to the probe to execute this command are the following:

Parameters	Description
interface	Network interface through which the packets from the PCAP file are sent.
filter	BPF filter used to select the packets to send from the PCAP file.
file	PCAP file containing the packets to be sent, encoded in Base64.

Table 27: PCAP execution parameters

The pcap command return no particular result, it sends a **success response message** with an empty result.

6 | Proof of concept of net.resolver

This section presents the proof of concept demonstrating the use of net.Resolver, as described in the implementation section 5.6.3 addressing the issue encountered during the configuration of the network interfaces. It begins with an introduction to the problem. The method used to configure the resolver is then presented, followed by an analysis of the results obtained. Finally, the section discusses the potential usefulness of this solution for future developments of the system.

Contents

6.1 Introduction	44
6.2 Explanation of the method	44
6.3 Results	45
6.4 Futur use	45

6.1 Introduction

This section of the report explains a proof of concept demonstrating the results obtained from the implementation of `net.resolver`.

As explained in the section 5.6.3 describing the issue encountered during the interface configuration, tests were conducted using `net.resolver`. Although it was not retained in the final implementation, the results obtained were rather positive. Since this approach could potentially be useful for the future development of the project, this section describes the implementation that was tested and the results obtained.

6.2 Explanation of the method

To implement the resolver, a **Resolver** object of type `*net.Resolver` was added to the probe. A `SetResolver` function was then created, which takes the IP address of the DNS server as a parameter.

The function was implemented as follows:

```
1 func (p *Probe) SetResolver(dnsServer string) {
2     log.Println("[PROBE] Setting Resolver with DNS Server", dnsServer)
3     p.Resolver = &net.Resolver{
4         PreferGo: true,
5         Dial: func(ctx context.Context, network, address string) (net.Conn, error) {
6             d := net.Dialer{
7                 Timeout: 5 * time.Second,
8             }
9             return d.DialContext(ctx, "udp", dnsServer+":53")
10        },
11    }
12 }
```

This function initializes the probe's resolver by creating a `net.Resolver`. The *PreferGo* attribute is enabled to force the use of Go's native DNS implementation. This allows the custom `Dial` function to be used instead of the DNS resolver provided by the operating system. The `Dial` function defines how connections to the DNS server are established. For each DNS request, a UDP connection is created to the specified server on port 53, which is the standard port used by the DNS protocol. A 5-second timeout is configured to prevent a blocked DNS request from indefinitely stopping the program.

By default, the libraries used in this project, such as the one used to perform ping operations, rely on the system's default DNS resolver. Therefore, a custom function had to be implemented to resolve hostnames to IP addresses using the probe's resolver before passing them to these libraries. The following function was implemented for this purpose:

```
1 func (p *Probe) ResolveIPv4(ctx context.Context, target string) (string, error) {
2     if ip := net.ParseIP(target); ip != nil {
3         if ipv4 := ip.To4(); ipv4 != nil {
4             return ipv4.String(), nil
5         }
6         return "", fmt.Errorf("IPv6 is not supported")
7     }
8 }
```

```

8
9  ips, err := p.Resolver.LookupIP(ctx, "ip", target)
10 if err != nil {
11     return "", err
12 }
13
14 for _, ip := range ips {
15     if ipv4 := ip.To4(); ipv4 != nil {
16         return ipv4.String(), nil
17     }
18 }
19
20 return "", fmt.Errorf("no IPv4 address found for %s", target)
21 }

```

The function first checks whether the target is already a valid IPv4 address. If so, the target is returned directly, as no DNS resolution is required. If the target is not an IPv4 address, it is considered to be a hostname. The function then performs a DNS resolution using the resolver previously configured in the probe. The *LookupIP* method sends a DNS request to retrieve the list of IP addresses associated with the hostname (target). Since a hostname can resolve to multiple IP addresses, the function iterates through the returned results and selects only IPv4 addresses. The first IPv4 address found is returned. If no IPv4 address is available among the DNS results, an error is returned.

6.3 Results

The DNS server address was configured by manually setting the IP address of the DNS server used on the network to which the probe was connected. The `/etc/resolv.conf` file was then cleared to ensure that Docker had no access to a DNS server and to properly test the implementation of `net.resolver`. A ping to `google.ch` and an HTTP request to Google were then performed to verify that the program could not resolve hostnames using the system DNS configuration. As expected, both operations failed. The `ResolveIPv4` function was then integrated into the functions responsible for executing ping and HTTP requests. The same tests were performed again. This time, both operations succeeded, demonstrating that the custom resolver was correctly resolving hostnames using the configured DNS server.

6.4 Futur use

The use of `net.resolver` could be adapted and reused in the future to configure a specific DNS server for each probe interface. If `eth0`, `eth1`, and `eth2` are connected to different networks, and each network provides its own DNS server, it may be necessary to configure a custom DNS resolver for each interface. Therefore, using one `net.resolver` per interface could be a potential approach for future development of the probes.

7 | Validation

This section presents the validation of the developed system to verify that the implemented functionalities meet the requirements defined previously. The different tests are performed using a dedicated test bench.

The validation is then divided into two main parts. First, functional tests are performed to verify that the implemented functionalities behave as expected and that the system requirements are fulfilled. Second, limit tests are conducted to identify the limitations of the probe and evaluate its behaviour as the workload increases. Finally, a conclusion summarizes the different tests performed and the main results obtained.

Contents

7.1 Testbench	47
7.2 Functionnal tests	48
7.3 Limit tests	50
7.3.1 Capture size limit	50
7.3.2 Parallel function limit	51
7.4 Conclusion	53

7.1 Testbench

To demonstrate and test the different capabilities of the probe under conditions close to its real-world use, a test bench was set up. It reproduces a test environment consisting of several network devices and makes it possible to perform various tests on a real device.

The test bench consists of three probes, **SEP0**, **Bridge**, and **LAN**, as well as a coordinator implementing the MQTT broker. It also includes two HOOC Connect XT devices. The first one serves as the DUT (Device Under Test), its the device on which the various tests are performed. The second HOOC Connect XT provides Internet access to the coordination network.

The test bench consists of two separate networks: the coordination network and the test network.

The coordination network uses the `10.0.0.0/24` subnet. It is used for communication between the coordinator and the different probes deployed in the test environment.

The test network corresponds to the networks on which the various tests are performed. The DUT separates this network into two parts. The SEP0 side uses the `192.168.2.0/24` subnet, while the LAN side uses the `192.168.3.0/24` subnet. Two probes are deployed on the SEP0 side: the SEP0 probe and the Bridge probe. On the LAN side of the DUT, only the LAN probe is deployed.

The diagram below presents the complete architecture of the test bench and the connections between the different devices.

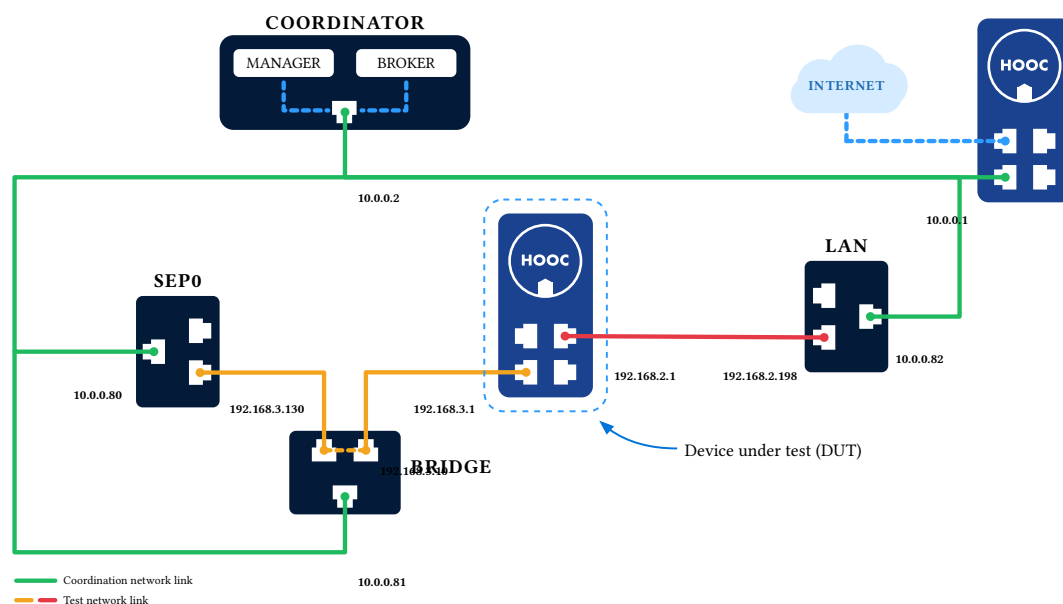


Figure 11: Test bench

The testbench was configured to reproduce the different network configuration scenarios that a probe may encounter in a test environment. Static IP addresses were assigned to the coordination network interfaces to simulate a test environment without a DHCP server. The Bridge Probe was configured to operate as a bridge between its test interfaces, with a static IP address assigned to the bridge interface. Similarly, a static IP address was assigned to the eth2 interface of SEP0. In contrast, the eth2 interface of the LAN Probe receives its IP address dynamically from a DHCP server provided by the HOOC. This configuration allows both static and dynamic IP address assignment to be tested within the same testbench.

7.2 Functionnal tests

Various functional tests were performed in the test bench to validate the operation of the probe and verify that all the requirements defined previously were correctly implemented and functional.

Parameter configuration and save

First, tests were performed to verify that configuration changes sent by the coordinator were correctly applied by the probe. Different parameters were modified through the coordinator, and their values were then checked on the probe to confirm that the changes had been correctly received and applied. The probe was then restarted to verify that the new configurations were properly saved and restored after a restart.

Interface configuration

Tests were also performed to validate the different network interface configurations supported by the probe. The static IP configuration was tested by manually assigning an IP address to an interface through the coordinator and verifying that the configuration was correctly applied. Dynamic configuration was also tested to verify that the probe's interfaces could automatically obtain an IP address through DHCP. The probe's ability to operate as a DHCP server on a test interface was also verified by connecting devices to the interface and checking that they could successfully obtain an IP address. Finally, the bridge mode was validated by configuring eth1 and eth2 as a bridge and verifying that network traffic could pass through the probe transparently. These tests confirmed that the network configurations required for the probe to operate in in both system environments were correctly supported.

Multi-environnement

Tests were also performed to validate the MQTT connection management in both environments. The probe was successively connected to the configuration broker and the test broker to verify that it could correctly detect and establish a connection with each of them. Disconnections were then intentionally triggered to verify that the probe detected the loss of connection and automatically attempted to re-establish communication with the available broker.

The probe was also moved from one environment to the other to verify that it could detect the network change and connect to the corresponding broker. These tests validated the probe's connection, reconnection, and broker switching mechanisms, which are essential for the probe to operate correctly across the two environments defined by the system.

Command testing

Functional tests were also performed for the different commands available on the probe. Several test suites were created directly from the coordinator's web interface to group and execute different test scenarios. Each command was tested with different parameters to verify that it could be correctly received and executed, and that the returned result matched the expected behavior. Invalid parameters were also used to verify the error handling mechanisms.

Dynamic definition

The dynamic definition of commands and configuration parameters by the probe was also tested through the coordinator's web interface. The objective was to verify that all metadata required to display and configure commands and parameters is provided by the probe itself. The information announced by the probe was correctly interpreted and displayed by the web interface.

During the development of the probe, new commands and configuration parameters were progressively implemented and added to the announced capabilities. Each of them appeared automatically in the web interface with the corresponding information, without requiring any modification to the coordinator. This validates the dynamic behavior of the interface and confirms that the probe provides all the metadata required for the coordinator to display and handle the probe capabilities and configuration parameters.

Summary of functional test

The functional tests presented in this section were used to validate the different requirements defined in section 4.2. The following table summarizes the requirements covered by the tests and the corresponding test section used for their validation.

Requirement	Validation
The probe registers with the coordinator and announces its capabilities and configuration parameters	Validated through the tests presented in Dynamic definition , which verify that the probe correctly registers with the coordinator and provides the metadata required to dynamically display its capabilities and configuration parameters.
The probe applies configuration changes and saves the configuration	Validated through the tests presented in Parameter configuration and save , which verify that configuration changes are correctly applied and preserved after a restart.
The probe supports the required network interface configurations	Validated through the tests presented in Interface configuration , covering static and dynamic IP configuration, DHCP client configuration, DHCP server configuration, and bridge mode.
The probe receives commands, executes them, and sends the corresponding results	Validated through the tests presented in Command testing , where the available commands are executed with valid and invalid parameters and their results and error handling are verified.
The probe identifies and connects to the appropriate broker	Validated through the tests presented in Multi-environment , which verify broker detection, connection, reconnection after a disconnection, and broker switching when moving between the two environments.

Table 28: Summary of functional test validations

7.3 Limit tests

In addition to the functional tests, several other tests were performed to evaluate the limits of the probe and observe its behavior under higher loads. The objective was to determine whether the probe remained functional when handling a large number of commands or network operations simultaneously, and to identify any performance limitations that could affect its use in a real environment.

7.3.1 Capture size limit

When the probe performs a network capture, the captured packets are temporarily stored in a PCAP file. Once the capture is completed, the content of this file is read, encoded in Base64 and sent to the coordinator before the temporary file is deleted.

Tests were performed to determine the maximum capture size that the probe can handle. For this purpose, iperf[16] was used to continuously generate network traffic between two probes. An iperf server was started on the SEP0 probe and an iperf client on the Bridge probe. A network capture was then started on the Bridge probe while the client continuously generated traffic. The size of the temporary PCAP file was monitored throughout the capture.

When the PCAP file size exceeded approximately **325 MB**, the probe process was terminated by the Linux kernel because of excessive memory consumption. This issue is caused by the way the PCAP file is prepared before being sent. First, the entire file is read into memory, resulting in an additional memory consumption approximately proportional to the file size. The data is then encoded into Base64, which requires an additional large memory allocation. Finally, the Base64-encoded data is included in the JSON response and serialized using `json.Marshal`, resulting in further memory allocations before the message is published through MQTT. Beyond approximately **325 MB** per PCAP file, the memory required to process and serialize the file for transmission can exceed the available memory of the hardware, causing the Linux kernel's OOM (Out-Of-Memory) killer to terminate the probe process.

It was also observed that the maximum message size supported by MQTT is smaller than the maximum PCAP file size that the probe can handle. Therefore, when transmitting a PCAP file to the coordinator, the limitation imposed by MQTT communication is reached before the probe's available RAM becomes a limiting factor. One possible solution for transmitting larger PCAP files would be to split the captured file into several parts and transmit them sequentially to the coordinator, which could then reassemble the parts to reconstruct the original file. This approach was not implemented as part of this thesis.

7.3.2 Parallel function limit

Tests were performed to determine the maximum number of functions that the probe can execute simultaneously. Different commands were used to evaluate the probe's behavior under increasing levels of parallelism.

First, load tests were performed to determine the maximum number of port scans that could be executed simultaneously by the probe. Port scans are performed using Nmap and can generate a significant load on system resources. For these tests, each port scan consisted of scanning all ports (1 to 65535) of the coordinator. The number of simultaneous port scans was progressively increased. The test started with a single port scan, and one additional port scan was added at each iteration to observe the evolution of the execution time and the behavior of the probe.

As the number of commands executed in parallel increased, the average execution time of the port scans also increased. This can be explained by the increasing workload placed on the processor, which must handle a larger number of port scans simultaneously. As the available processing resources are shared among more simultaneously portscans, the time required to complete each individual port scan increases.

Up to 23 simultaneous port scans, the execution times remained relatively consistent between the different scans. However, when more than 23 port scans were executed simultaneously, significant differences in execution time between individual scans were observed. This behavior is illustrated in the following graph.

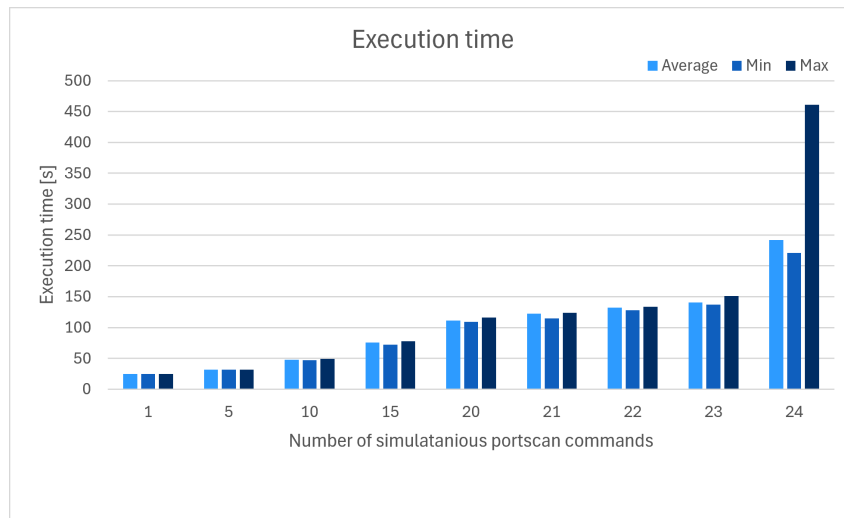


Figure 12: Evolution of the execution time

As shown in the graph, when 24 port scans are executed simultaneously, the maximum execution time of a port scan becomes significantly higher than the average execution time. Additional tests were performed by further increasing the number of concurrent port scans. These tests showed a recurring behavior: a group of approximately 23–24 port scans is executed simultaneously with similar execution times, while the execution times of the remaining scans become much less predictable and can be significantly longer. This behavior indicates that the probe reaches a limit in its ability to handle a high number of concurrent port scans consistently. Executing more than 23 port scans simultaneously is not recommended. Although the probe is still able to execute the commands, the execution time becomes significantly longer and less predictable, which reduces the efficiency of the probe.

Load tests were also performed to determine the number of **ping** commands that could be executed **simultaneously** by the probe. The objective was to identify the level of parallelism at which the probe begins to experience packet loss. Each ping command was targeting the IP address of the coordinator and had a ping count of 100. The ping count was fixed at 100 so the commands takes more time to execute. The number of simultaneous ping commands was then progressively increased. With **100 requests per command**, no packet loss was observed up to approximately **200 simultaneous commands**. Above this level, packet loss began to appear.

To evaluate the influence of the number of pings performed by each command, the number of pings was then reduced from 100 to 10. With this configuration, no packet loss was observed even with **300 simultaneous commands**. However, the execution time of the commands began to increase at around 250 parallel commands. With 300 simultaneous commands, some commands required more than twice the expected execution time. This indicates a degradation in the probe's performance despite the absence of packet loss.

These tests show that the probe is capable of executing a large number of ping commands in parallel while maintaining generally stable and reliable behavior.

Tests were also performed with TCP and UDP commands. A total of **200 commands** were executed in parallel, consisting of **100 TCP** commands and **100 UDP** commands. All commands were configured with the waiting-response parameter enabled. The tests completed successfully, with execution times ranging from approximately a few tens of milliseconds to 150 ms. This variation is considered acceptable for this type of test. As observed with the ping and port scan tests, execution times tend to increase as the number of commands executed in parallel increases. The tests were not continued beyond 200 simultaneous commands, as this level of parallelism is considered higher than what would be required in a real-world use of the probe. Performing additional tests at even higher levels of parallelism was therefore not considered particularly relevant.

7.4 Conclusion

The functional and limit tests performed throughout the development allowed the main requirements defined for the probe to be validated. The tests confirmed that the probe can communicate correctly with the coordinator, manage connections to both system environments, receive and apply configuration changes, execute the implemented commands, and return the expected results. The different network configuration mechanisms, including static and dynamic IP configuration, DHCP server operation, and bridge mode, were also successfully validated.

The limit tests also identified some constraints, particularly regarding the maximum size of PCAP captures and the performance degradation caused by a very high number of commands running in parallel.

Overall, all the objectives defined for this thesis were successfully achieved, and the implemented probe meets the requirements established for the project.

8 | Conclusion

8.1 Project summary

During this Bachelor's thesis, an embedded probe was designed and implemented to perform network tests in OT environments. The probe communicates with a coordinator through MQTT, receives the commands to be executed, performs the requested operations, and returns their results. It supports several types of network tests, including ICMP connectivity tests, TCP and UDP communications, HTTP requests, port scans, network traffic capture, and PCAP file replay.

The probe is remotely configurable, allowing its network interfaces and operating parameters to be adapted to different test environments. The configured parameters are saved locally in a configuration file, ensuring that they are preserved after a restart. The probe was also designed to operate in two distinct environments: a configuration environment and a test environment. This allows it to be prepared in a known environment before deployment and used in different network configurations without requiring extensive manual configuration in the OT environment.

The probe's operation was validated through various functional tests performed in a dedicated test bench, allowing its different functionalities to be verified in a representative environment. Performance tests were also conducted to evaluate its behavior when executing multiple operations in parallel and to identify its limits in terms of resource usage and execution time.

8.2 Comparison with the initial objectives

All requirements defined in Section 4.2 were successfully met. The probe can register with the coordinator and provide a complete list of its available commands and configuration parameters, together with the metadata required to display them in the web interface. Configuration parameters can be remotely modified by the coordinator and are stored persistently, allowing them to be preserved after a probe restart. The probe can also receive commands, execute the corresponding operations, and return their results to the coordinator. Execution logs are displayed directly in the terminal, providing visibility into the probe's activity and the commands being executed.

The probe can also identify the environment in which it is operating and automatically connect to the corresponding MQTT broker. Its network interfaces can be configured according to the requirements of the test environment, using either static or dynamically assigned IP addresses. In addition, the two interfaces dedicated to the test network can be configured in bridge mode, allowing the probe to be placed between two devices while transparently forwarding their network traffic.

8.3 Encountered difficulties

One of the difficulties encountered was that when a static IP address was assigned to an interface that had previously obtained an address through DHCP, the existing DHCP address remained active instead of being replaced. As a result, the interface had two IP

addresses simultaneously: one assigned by DHCP and the other configured statically. This behavior was caused by the OS DHCP client, which continued running in the background and automatically assigned an address to the interface. The difficulty came from the fact that the probe runs inside a Docker container, while the DHCP client belongs to the host OS, making it difficult to control it directly from the container. After several investigations, `nsenter` was identified as a solution, as it allows commands to be executed within the namespaces of another process, including its network namespace. This made it possible for the probe to execute the DHCP desactivation command within the host's network namespace and correctly disable the DHCP client on the targeted interface.

I also encountered difficulties related to DNS configuration, as described in Section 5.6.3. Another difficulty was related to the management of the two MQTT brokers used in the different environments. The probe had to be able to dynamically reconfigure its MQTT client depending on which broker was available. Implementing this mechanism required several adjustments to the MQTT client to ensure that the connection could be reliably established, interrupted, and restored when switching between environments.

8.4 Future perspectives

Several improvements could be considered for future versions of the probe:

- **Extended network configuration:** The current network configuration could be expanded with additional options to provide greater flexibility when adapting the probe to different test environments. For example, gateway configuration could be added to allow each interface to use a specific gateway when required. Another possible improvement would be to support the configuration of a specific DNS server for each interface. The `net.Resolver` implementation developed during the proof of concept could be further integrated for this purpose, allowing DNS requests to be performed through the appropriate network interface and DNS server. This would make the probe better suited to test environments with different network configurations and routing requirements.
- **PCAP file transfer in multiple parts:** As shown by the limitation tests, the size of a PCAP capture that can be transferred via MQTT is limited. Consequently, captures exceeding this limit cannot currently be sent to the coordinator. A possible improvement would be to split the PCAP file into smaller parts and transmit them separately. This would allow larger captures to be transferred and retrieved while remaining within the limitations of the MQTT protocol.
- **Additional test capabilities:** Another possible improvement would be to add new types of tests to make the probe more complete and increase the range of operations it can perform. This would allow the probe to cover more testing scenarios and evaluate a wider range of network and system properties in OT environments.

Glossary

Concepts

IoT – Internet of Things: Network of physical devices capable of collecting, exchanging, and processing data.

OT – Operational Technology: Systems and technologies used to monitor and control physical processes in industrial environments.

Data Formats

JSON – JavaScript Object Notation: Lightweight text-based format used to represent and exchange structured data.

MQTT

LWT – Last Will and Testament: MQTT mechanism that allows a message to be published when a client unexpectedly disconnects.

Networking

BPF – Berkeley Packet Filter: Packet filtering mechanism used to select network traffic matching specific criteria.

IP – Internet Protocol: Network-layer protocol responsible for addressing and routing packets between networked devices.

PCAP – Packet Capture: File format and mechanism used to store captured network packets for later analysis.

Protocols

DHCP – Dynamic Host Configuration Protocol: Protocol used to automatically assign network configuration parameters such as IP addresses to devices.

DNS – Domain Name System: System used to resolve domain names into IP addresses.

HTTP – HyperText Transfer Protocol: Application-layer protocol used to transfer data between clients and web servers.

ICMP – Internet Control Message Protocol: Network-layer protocol used for control messages and diagnostic operations such as ping.

MQTT – Message Queuing Telemetry Transport: Lightweight publish-subscribe messaging protocol used for communication between the probes and the coordinator.

TCP – Transmission Control Protocol: Connection-oriented transport protocol that provides reliable and ordered delivery of data.

UDP – User Datagram Protocol: Connectionless transport protocol that provides low-overhead data transmission without guaranteeing delivery.

Testing

DUT – **Device Under Test**: Device or system on which tests are performed to evaluate its behaviour and performance.

University

HEI – School of Engineering

IT – Infotronics

SYND – Systems Engineering

Bibliography

- [1] FriendlyELEC, “NanoPi R5S.” [Online]. Available: https://wiki.friendlyelec.com/wiki/index.php/NanoPi_R5S
- [2] Hardkernel Co., Ltd., “ODROID-H4.” [Online]. Available: <https://www.hardkernel.com/shop/odroid-h4/>
- [3] Hardkernel Co., Ltd., “NetCard 3 for H-series.” [Online]. Available: <https://www.hardkernel.com/shop/netcard3-for-h-series/>
- [4] Eclipse Foundation, “Eclipse Paho MQTT Go Client.” Accessed: Aug. 11, 2026. [Online]. Available: <https://pkg.go.dev/github.com/eclipse/paho.mqtt.golang>
- [5] A. Vishvananda, “Netlink library for Go.” [Online]. Available: <https://pkg.go.dev/github.com/vishvananda/netlink>
- [6] Dnsmasq, “Dnsmasq: A lightweight server.” [Online]. Available: <https://thekelleys.org.uk/dnsmasq/doc.html>
- [7] Prometheus Community, “pro-bing.” [Online]. Available: <https://pkg.go.dev/github.com/prometheus-community/pro-bing>
- [8] gopacket, “GoPacket.” [Online]. Available: <https://github.com/gopacket/gopacket>
- [9] Gopacket, “Package pcap.” [Online]. Available: <https://pkg.go.dev/github.com/gopacket/gopacket/pcap>
- [10] Gopacket, “Package pcapgo.” [Online]. Available: <https://pkg.go.dev/github.com/gopacket/gopacket/pcapgo>
- [11] Ullaakut, “nmap Go library.” [Online]. Available: <https://pkg.go.dev/github.com/Ullaakut/nmap/v4>
- [12] The Go Authors, “Package http.” [Online]. Available: <https://pkg.go.dev/net/http>
- [13] The Go Authors, “Package base64.” [Online]. Available: <https://pkg.go.dev/encoding/base64>
- [14] The Go Authors, “Package utf8.” [Online]. Available: <https://pkg.go.dev/unicode/utf8>
- [15] The Go Authors, “Package net.” [Online]. Available: <https://pkg.go.dev/net>
- [16] iPerf, “iPerf Documentation.” [Online]. Available: <https://iperf.fr/iperf-doc.php>

A | Appendix

AA Deployment

AAA `Dockerfile` used to build the image

Here is the `Dockerfile` used to create the probe image

```
1 FROM golang:1.26 AS deps
2
3 WORKDIR /app
4
5 RUN apt-get update && apt-get install -y libpcap-dev && rm -rf /var/lib/apt/lists/*
6
7 COPY go.mod go.sum ./
8 RUN --mount=type=cache,target=/go/pkg/mod go mod download
9
10 FROM deps AS build
11 COPY . .
12
13 RUN --mount=type=cache,target=/go/pkg/mod go build -o /out/probeBin .
14
15 FROM debian:bookworm-slim AS probe
16
17 RUN apt-get update && apt-get install -y libpcap0.8 ca-certificates nmap dnsmasq procs
    util-linux && rm -rf /var/lib/apt/lists/*
18
19 WORKDIR /
20 COPY --from=build /out/probeBin /probe/probe
21
22 RUN mkdir -p /probe/config
23
24 ENTRYPOINT ["/probe/probe"]
```

AAB `docker-compose.yml`

Here is the `docker-compose.yml` used to deploy the probe image:

```

1 services:
2   probe:
3     image: ghcr.io/paulojun-gomescos/code-probe:latest
4     build:
5       context: .
6       dockerfile: Dockerfile
7     platform: "linux/arm64"
8     working_dir: /
9     environment:
10      - PROBE_CONFIG_PATH=${PROBE_CONFIG_PATH}
11      - MQTT_BROKER=${MQTT_BROKER}
12      - MQTT_USERNAME=${MQTT_USERNAME}
13      - MQTT_PASSWORD=${MQTT_PASSWORD}
14     volumes:
15      - probe_data:/probe/config
16     restart: unless-stopped
17     network_mode: host
18
19 volumes:
20   probe_data:

```

AAC `.env` file (environment variables)

Here is the `.env.example` file, which documents the `.env` file required for the probe deployment :

```

1 # Path to the file containing the probe configuration
2 PROBE_CONFIG_PATH=probe/config/probeConfig.json
3
4 # MQTT credentials for the configuration broker
5 MQTT_BROKER=UrlBrokerConfig
6 MQTT_USERNAME=UsernameBrokerConfig
7 MQTT_PASSWORD=PasswordBrokerConfig

```

AAD Deployment guide

Before deploying the probe, Docker and Docker Compose must be installed on the NanoPi R5S. The device must also have network connectivity to the container registry in order to pull the probe image. Once these prerequisites are installed and the device is connected to the required networks, the probe can be deployed using the following steps.

The following steps describe how the probe is deployed on the NanoPi R5S.

1. Connect to the NanoPi R5S using SSH:

```
ssh root@<NanoPi_IP_Address>
```

```
password: fa
```

1. Create a “probe” directory on the NanoPi R5S
2. Add the `docker-compose.yml` file to the directory
3. Copy the `.env.example` in the `.env` file and fill in the MQTT credentials of the broker of the configuration environment
4. Pull the docker image from `ghcr.io`

```
1 docker compose pull
```

 Shell

5. Start the probe container

```
1 docker compose up -d
```

 Shell